



National School of Computer Science and Systems Analysis (ENSIAS), Rabat

Summer Internship Report

Program: Cybersecurity, Cloud and Mobile Computing

CyberSOC AI

An Intelligent Multi-Application Defensive Monitoring Platform

---

Completed by:

Ms. Salma BOULHANA

Supervised by:

Mr. Smail El Faiz

Defended on **Tuesday, 27th October 2026**, before the honorable jury:

Pr. Jamal EL HACHMI
Pr. Moulay Ahmed FAQIHI

Academic Year: 2025–2026

Acknowledgments

First and foremost, we thank Allah for giving us the strength, patience, and perseverance to complete this work.

We express our sincere gratitude to our company supervisor, **Mr. Smail El Faiz**, for his guidance, availability, and practical advice throughout the internship. His support helped us understand the company's needs and move the project forward.

We also thank our academic supervisors and the ENSIAS teaching staff for the knowledge and support they provided during our studies. We are grateful to the jury members, **Mr. Jamal EL HACHMI** and **Mr. Moulay Ahmed FAQIHI**, for agreeing to evaluate this work and for the time devoted to its review.

Our thanks go to the **MARTECH Conseils** team for welcoming us and helping us understand the company's technical environment. We also thank everyone who contributed, directly or indirectly, to this project.

Abstract

Organizations rely on applications, servers, and networks that produce large amounts of security data. These records are often collected by different tools and stored in different formats. As a result, reviewing them separately can make it difficult to see suspicious activity across several systems and decide which events need attention first. This internship project addressed this need through the development of CyberSOC, a platform for centralizing and reviewing security events from multiple monitored environments.

The platform connects security monitoring tools with a web application. Wazuh, Suricata, Docker logs, and OWASP ZAP provide security events or scan findings. The FastAPI backend processes and normalizes supported data, classifies incidents, and stores platform records in PostgreSQL. A Next.js dashboard lets administrators manage the platform, SOC analysts review security findings, and readers consult information with limited access. The application also includes security scoring, report generation, and an optional recommendation service that can use the Gemini API when configured.

The project followed a mixed methodology. Requirements, analysis, and design were carried out in sequence using a Waterfall approach. Development, testing, and validation then proceeded iteratively. The SSH anomaly detection service uses an Isolation Forest model trained separately and saved as a model file. The service loads that model and uses it to score SSH event windows. It does not train a new model for each incoming event.

Testing was performed in the available project environment using the documented scenarios and replayed logs. The ZAP test evidence reports seven alerts received, six events imported, and three incidents created. Other screenshots show authentication-related and Fail2ban incidents displayed in the dashboard. These results demonstrate the event-import and presentation flow in the test setup. They do not establish production-scale performance or detection accuracy.

CyberSOC brings several security functions into one interface and provides a starting point for centralized monitoring. Further work should include automated regression tests, evaluation with larger and representative datasets, performance measurements, and validation with authorized live sources.

Keywords: Cybersecurity monitoring, SOC, security events, incident management, anomaly detection, Wazuh, Isolation Forest.

Résumé

Les organisations utilisent des applications, des serveurs et des réseaux qui produisent une grande quantité de données de sécurité. Ces informations sont souvent collectées par des outils différents et enregistrées dans des formats variés. Leur consultation séparée peut rendre difficile l'identification d'activités suspectes concernant plusieurs systèmes et la priorisation des événements à examiner. Ce projet de stage répond à ce besoin par le développement de CyberSOC, une plateforme de centralisation et d'analyse des événements de sécurité provenant de plusieurs environnements supervisés.

La plateforme relie des outils de supervision à une application web. Wazuh, Suricata, les journaux Docker et OWASP ZAP fournissent des événements ou des résultats d'analyse. Le backend FastAPI traite et normalise les données prises en charge, classe les incidents et enregistre les informations de la plateforme dans PostgreSQL. Un tableau de bord Next.js permet aux administrateurs de gérer la plateforme, aux analystes SOC d'examiner les événements de sécurité et aux lecteurs de consulter les informations avec des droits limités. L'application comprend également un score de sécurité, la génération de rapports et un service de recommandations optionnel qui peut utiliser l'API Gemini lorsqu'elle est configurée.

Le projet a suivi une méthodologie mixte. Les besoins, l'analyse et la conception ont été réalisés dans l'ordre selon une démarche Waterfall. Le développement, les tests et la validation ont ensuite progressé de manière itérative. Le service de détection d'anomalies SSH utilise un modèle Isolation Forest entraîné séparément et enregistré dans un fichier. Le service charge ce modèle et l'utilise pour analyser des fenêtres d'événements SSH. Il ne réentraîne pas un nouveau modèle pour chaque événement reçu.

Les essais ont été réalisés dans l'environnement disponible à partir des scénarios documentés et de journaux rejoués. Les résultats du test OWASP ZAP indiquent sept alertes reçues, six événements importés et trois incidents créés. D'autres captures montrent des incidents liés à l'authentification et à Fail2ban dans le tableau de bord. Ces résultats illustrent le parcours d'importation et d'affichage des événements dans l'environnement de test. Ils ne permettent pas de conclure sur les performances en production ni sur la précision globale de la détection.

CyberSOC regroupe plusieurs fonctions de sécurité dans une interface unique et constitue une première base pour la supervision centralisée. Les prochaines étapes devraient inclure des tests de régression automatisés, une évaluation sur des jeux de données plus grands et représentatifs, des mesures de performance et une validation avec des sources réelles autorisées.

Mots-clés : Supervision cybersécurité, SOC, événements de sécurité, gestion des incidents, détection d'anomalies, Wazuh, Isolation Forest.

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DAST	Dynamic Application Security Testing
ECS	Elastic Common Schema
EVE	Extensible Event Format (Suricata EVE JSON)
HTTP	Hypertext Transfer Protocol
IDS	Intrusion Detection System
OSSEC	Open Source Security Event Correlator
IP	Internet Protocol
IPS	Intrusion Prevention System
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
JSONB	JSON Binary
JWT	JSON Web Token
ML	Machine Learning
MVP	Minimum Viable Product
OWASP	Open Worldwide Application Security Project
PDF	Portable Document Format
RBAC	Role-Based Access Control
REST	Representational State Transfer
SIEM	Security Information and Event Management
SPA	Single-Page Application
SOC	Security Operations Center
SQL	Structured Query Language
SSH	Secure Shell
UML	Unified Modeling Language
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
XDR	Extended Detection and Response
ZAP	Zed Attack Proxy (OWASP ZAP)

Contents

Acknowledgments **i**
Abstract **ii**
Résumé **iii**
List of Abbreviations **iv**
List of Figures **vii**
List of Tables **viii**
General Introduction **1**
1 Chapter 1: General Context of the Project **2**
Introduction . . . 2
1 Host Organization . . . 2
2 Project Context . . . 3
3 Project Management Approach . . . 3
Conclusion . . . 5
2 Chapter 2: Requirements Analysis **6**
Introduction . . . 6
1 Identification of Actors . . . 6
2 Functional Requirements . . . 7
3 Non-Functional Requirements . . . 8
4 Project Constraints . . . 8
5 Use-Case Analysis . . . 8
Conclusion . . . 13
3 Chapter 3: Technology Study and Design **14**
Introduction . . . 14
1 Fundamental Concepts . . . 14
2 Technology Study . . . 15
3 Global Architecture . . . 17
4 Event Processing Design . . . 19
5 Class Diagram . . . 21
6 Application Registration and Source Mapping . . . 22
7 Main Scenarios . . . 23
8 Detection, Scoring and Recommendation Design . . . 25
9 Security Design . . . 26
Conclusion . . . 26
4 Chapter 4: Implementation **27**
Introduction . . . 27
1 Development Environment . . . 27
2 Technology Stack . . . 27
3 Installation, Configuration and Deployment . . . 28
4 Implementation by Phase . . . 30
5 Presentation of the Platform . . . 33

6 Technical Challenges and Solutions . . . 35
Conclusion . . . 36
5 Chapter 5: Testing and Validation **37**
Introduction . . . 37
1 Validation Approach . . . 37
2 Functional Test Scenarios . . . 37
3 Integration Test of the Full Pipeline . . . 39
4 Detection Tests with Replayed Logs . . . 39
5 Tests of the Machine Learning Engine . . . 40
6 Security and Reliability Tests . . . 41
7 Results and Discussion . . . 41
Conclusion . . . 42
General Conclusion **43**
Bibliography **44**
Image Credits **44**
Appendices: Project Files **45**

List of Figures

1.1 Mixed Waterfall and iterative methodology used for the project 4
1.2 Planned schedule for the internship 5

2.1 Global use-case diagram of CyberSOC 10
2.2 Use cases for managing monitored applications 11
2.3 Use cases for collecting and analyzing events 11
2.4 Use cases for incident analysis and security scoring 12
2.5 Use cases for recommendations and model maintenance 12
2.6 Use cases for the SOC dashboard 12
2.7 Use cases for security reports 13

3.1 High-level architecture of the platform 18
3.2 Docker deployment diagram 18
3.3 Activity diagram of the event processing pipeline 21
3.4 Class diagram of the platform 22
3.5 Sequence diagram for analyzing a brute-force alert 24
3.6 Sequence diagram for generating a PDF report 24
3.7 Sequence diagram for adding a monitored application 25

4.1 Administrator dashboard 33
4.2 Incident list in CyberSOC 33
4.3 User management page 34
4.4 OWASP ZAP scan results 34
4.5 PDF reports page 35

5.1 Wazuh alert import during Phase 2 testing 38
5.2 OWASP ZAP findings imported during Phase 3 testing 38
5.3 Evidence from the end-to-end Wazuh test 39
5.4 Incidents displayed after Fail2ban log replay 40
5.5 Authentication incidents displayed after log replay 40

List of Tables

1.1 Main phases of the project 4

2.1 Main human actors of CyberSOC 6
2.2 External systems and monitored resources 7
2.3 Selected functional requirements 7
2.4 Selected non-functional requirements 8
2.5 Main project constraints 8

3.1 Evaluation criteria and weights 15
3.2 Weighted scores of the candidate technologies 16
3.3 Technology stack retained 17
3.4 Docker networks and associated containers 19
3.5 Output formats of the security tools 19
3.6 Fields of the normalized event schema 20
3.7 Conversion of the severity levels by source 20
3.8 Planned access rights by role 26

4.1 Services defined in the Docker Compose file 29
4.2 Setup checks and available evidence 30
4.3 Phase 2 tasks and validation 31
4.4 Phase 3 tasks and validation 31
4.5 Phase 4 tasks and validation 32
4.6 Wazuh risk value and severity mapping 32

5.1 Incremental behavior of the source collectors 41
5.2 Validation of the requirements 42

General Introduction

Organizations depend on applications, servers, containers, and networks to deliver their services. These systems produce security data every day: authentication records, application logs, network alerts, and results from security scans. The information is useful, but it is often spread across several tools. When each source is reviewed separately, it takes more effort to connect related events and decide which ones need attention first.

During our summer internship at MARTECH Conseils, under the professional supervision of Mr. Smail El Faiz, we developed CyberSOC, a defensive monitoring platform for several applications and technical environments. MARTECH Conseils did not have an existing Security Operations Center (SOC) platform of this kind. The project therefore focused on building a first platform to centralize and present security information, rather than extending an already operating SOC.

The main problem addressed by the project is how to collect events from different sources, organize them in a common form, identify incidents, and make the results understandable to users with different responsibilities. The platform connects security tools such as Wazuh, Suricata, Docker, and OWASP ZAP to a web application. It includes application and user management, event normalization, incident classification, a Security Score, recommendations, reports, and an SSH anomaly detection service. Three application connection methods are represented in the data model: Docker containers, base URLs, and Wazuh agent identifiers. Testing with Wazuh agents was completed; testing of URL-based applications remained incomplete.

We followed a mixed project methodology. The early work on understanding the need, requirements, and design followed a sequential Waterfall approach. Development and validation then advanced through iterative cycles, so features could be implemented, tested, reviewed, and adjusted as the project progressed. The SSH anomaly detector uses a model trained separately and saved for later use. The running service loads that model to analyze event windows; it does not train a new model for each input.

This report is organized into five chapters. **Chapter 1** introduces MARTECH Conseils, the project context, objectives, team organization, and methodology. **Chapter 2** presents the platform actors, scope, and main functional and non-functional requirements. **Chapter 3** explains the selected technologies and the system design, including the event pipeline, application connection methods, data model, and Security Score. **Chapter 4** describes the implementation and the platform's main functions. **Chapter 5** details the tests, observed results, and remaining validation limits. The general conclusion then reviews the contribution and identifies practical next steps.

Chapter 1: General Context of the Project

Introduction

The general introduction presented the need to bring security information from different sources into one platform. This chapter introduces MARTECH Conseils, explains the context and objectives of CyberSOC, and summarizes how we organized the project. The next chapter turns these needs into system requirements.

1 Host Organization

This section presents MARTECH Conseils, the ManagTech software environment, and the context of our internship. MARTECH provides technical and security solutions, while ManagTech presents software and digital services for business management.

1.1 MARTECH Conseils

MARTECH Conseils is a Moroccan technology company based in Rabat. Its services cover several areas related to technical infrastructure, security, and energy. The company describes an approach that can include understanding customer needs, planning a solution, installing it, and providing technical support [1].

Its main activities include:

- **Networking:** computer network equipment, connectivity, and installation for homes and organizations.
- **Electronic security:** video surveillance, alarms, access control, and related systems used to protect buildings and manage access.
- **Electrical systems:** electrical equipment and installations adapted to customer requirements.
- **Solar energy:** solar solutions intended to support energy needs and infrastructure projects.
- **Automation and home automation:** automated doors, connected access systems, and technologies for controlling home or work environments.
- **Project support:** technical guidance during project planning, installation, and follow-up.

These activities combine physical equipment with digital and connected technologies. This context is relevant to CyberSOC because security monitoring concerns both applications and the technical environments on which they operate.

1.2 ManagTech

The software activities relevant to our internship are presented through ManagTech, the business software environment associated with MARTECH Conseils. The company presents a configurable solution for managing public procurement submissions and related documents. It also highlights validation settings and access-right management [2].

The ManagTech environment supports business processes through features such as:

- **Document management:** organizing information and documents related to business submissions.
- **Validation settings:** adapting document checks and approval steps to working needs.
- **Access rights:** controlling which users can access or manage information.
- **Configurable workflows:** adapting the software to the organization's way of working.

Together, MARTECH's technical activities and ManagTech's software services provide the professional context for our project. CyberSOC was developed during our internship under the supervision of Mr. Smail El Faiz. Before this project, MARTECH did not have this SOC platform. We worked as a team of three students: Salma Boulhana, Hiba Rkhis, and Khadija Lamhailik. Our work focused on developing a platform to centralize security events and support the monitoring of applications and technical environments.

2 Project Context

Applications, servers, and containers produce security logs in different formats. Wazuh, Suricata, Docker, and OWASP ZAP each provide useful information, but reviewing their results separately makes it harder to identify and prioritize incidents. The project therefore aimed to develop a platform that brings this information together and presents it in a form that is easier to review.

The main objectives were to:

- collect and normalize security events from the supported sources;
- detect and classify suspicious activity;
- calculate a Security Score globally and for each application;
- present incidents and recommendations in a role-based dashboard;
- generate PDF reports.

The project was designed for several application types. An application could be associated with a Docker container, a Wazuh agent, or a base URL for security analysis. The Wazuh-agent and Docker-container paths were tested; end-to-end testing of the URL-based path remained incomplete.

3 Project Management Approach

This section presents the methodology used for the project, how we organized and coordinated the team's work, and how the internship was planned. It also summarizes the project phases, schedule, scope, and main deliverables.

3.1 Methodology

We used a mixed methodology. Requirements analysis and system design followed a planned sequence. Development and validation then progressed through iterations, allowing us to test features, address integration issues, and adjust the work as needed. Figure 1.1 summarizes this approach.

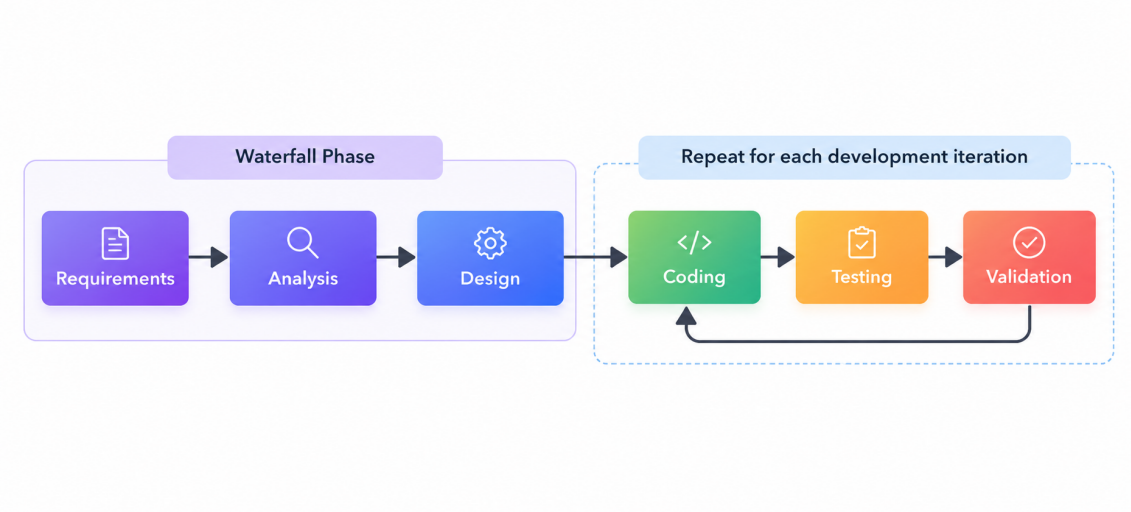


Figure 1.1: Mixed Waterfall and iterative methodology used for the project

3.2 Team Organization and Planning

We coordinated our work around the shared project goals and integrated features as they became ready. Git branches helped us manage changes to the shared code. The initial task plan provided a guide, but we adjusted the order of tasks when technical dependencies or test results required it.

The internship ran from 21 June to 21 August 2026. Table 1.1 summarizes the project phases, and Figure 1.2 shows their planned schedule. The schedule was adapted during the work, so it should be read as a guide rather than a fixed record of every task.

Table 1.1: Main phases of the project

Phase	**Approach**	**Main work**
Phase 0	Waterfall	Study, requirements, and tool selection
Phase 1	Waterfall	Architecture, data model, and environment setup
Phases 2–4	Iterative	MVP development, security integrations, testing, and monitoring

Figure 1.2 presents the planned tasks across the internship period.

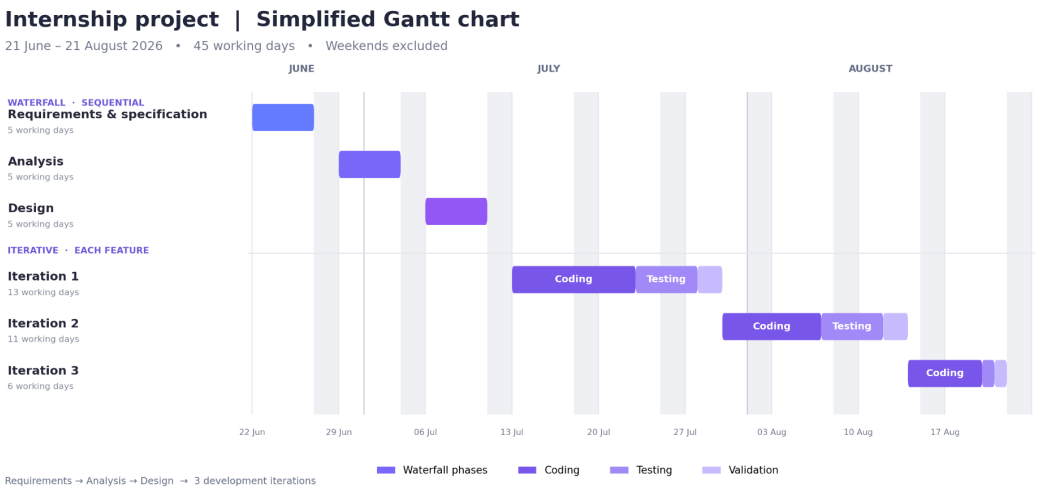


Figure 1.2: Planned schedule for the internship

The first version focused on the core event-processing workflow, user roles, incident display, scoring, and reporting. Later iterations added or extended integrations and monitoring features. Some planned work was adjusted as development and testing progressed.

Conclusion

This chapter introduced MARTECH Conseils, the reason CyberSOC was developed, and the main project objectives. It also summarized the team's work organization, the mixed methodology, and the internship schedule. The next chapter defines the platform's actors and requirements in more detail.

Chapter 2: Requirements Analysis

Introduction

Chapter 1 presented MARTECH Conseils, the project objectives, and the way we organized our work. This chapter defines the people and systems that interact with CyberSOC, summarizes its main requirements, and presents the use cases and scenarios that guided its design. Chapter 3 will use these requirements to explain the platform architecture and technical choices.

1 Identification of Actors

CyberSOC is intended for three main user roles. It also exchanges information with security tools and monitored environments. Table 2.1 summarizes the users and their main responsibilities.

Table 2.1: Main human actors of CyberSOC

Actor	**Access**	**Main responsibilities**
Administrator	Full access	Manage user accounts, roles, and monitored applications.
SOC Analyst	Operational access	Review events and incidents, follow their severity, consult recommendations, and generate reports.
Reader / Auditor	Read-only access	Consult dashboards, incident history, and available reports.

These roles are enforced through authentication and role-based access control. A technical maintainer may also interact with the security tools and deployment environment, but this is not one of the three application roles defined for the platform.

Security products and monitored resources act as external system actors because they provide data to CyberSOC. Table 2.2 lists the main sources considered in the project.

Table 2.2: External systems and monitored resources

System	Type	Role in the project
Wazuh	Host security monitoring	Supplies security alerts from monitored hosts and agents.
Suricata	Network detection tool	Produces network security alerts.
OWASP ZAP	Web application scanner	Produces scan findings for web applications.
Docker Engine	Container platform	Provides information and logs related to monitored containers.
Monitored applications	Applications and APIs	Represent the environments whose security events are monitored. Application records can include a container name, a base URL, or a Wazuh agent identifier, depending on the integration.

2 Functional Requirements

Functional requirements describe the services expected from the platform. We present the main requirements in Table 2.3; it is a focused summary rather than a repetition of every detailed requirement from the initial specification.

Table 2.3: Selected functional requirements

ID	Area	Requirement
RF01	Application management	Allow an administrator to register and manage monitored applications and their available connection details.
RF02	Event collection	Collect security events from integrated sources, including Wazuh.
RF03	Normalization	Convert incoming events into a common format that can be stored and analyzed.
RF04	Detection	Identify relevant suspicious activity, such as repeated failed logins or other configured event patterns.
RF05	Incident analysis	Classify incidents by severity and calculate a security score from the collected information.
RF06	Recommendations	Provide a recommendation associated with an incident, using the available rules or model output.
RF07	Dashboard	Let authorized users consult incidents, scores, and their history.
RF08	Reports and access	Support report generation and restrict platform features according to the user's role.

The monitored-resource record can represent a container, a URL, or a Wazuh agent. These are supported target types in the project model; the level of integration and testing for each

type is described in Chapter 5. The requirements therefore describe the intended platform and should not be read as a claim that every connector was fully tested.

3 Non-Functional Requirements

Non-functional requirements describe how the platform should operate. We selected the most relevant quality requirements for this project; Table 2.4 summarizes them.

Table 2.4: Selected non-functional requirements

Quality	**Expected requirement**
Security	Authenticate users, apply role-based permissions, and protect sensitive configuration values.
Reliability	Validate incoming data and handle unavailable or invalid source data without corrupting stored records.
Maintainability	Keep collection, analysis, reporting, and user-interface functions organized as separate components.
Usability	Present incidents and their severity in a clear way so that analysts can review them efficiently.
Portability	Use container-based deployment to make the development environment easier to reproduce.

4 Project Constraints

The requirements were defined within a limited internship period and a development environment intended for testing rather than production use. The main constraints are summarized in Table 2.5.

Table 2.5: Main project constraints

Constraint	**Effect on the project**
Time	The work had to be planned and prioritized within the internship dates.
Integration	The behavior and availability of external tools could affect collection and testing.
Test environment	Tests were carried out in a controlled environment; they do not represent continuous production monitoring.
Scope	Features and integrations were developed progressively, so the final implementation does not necessarily cover every initial expectation.

5 Use-Case Analysis

This section shows how the actors use CyberSOC. We first present the overall view, then the use cases for each main module, and finally three sequence diagrams for event analysis,

report generation, and application registration. The diagrams describe the intended interactions; implementation and test results are discussed in later chapters.

5.1 Global Use-Case Diagram

Figure 2.1 gives a high-level view of the actors, security sources, and main services offered by CyberSOC.

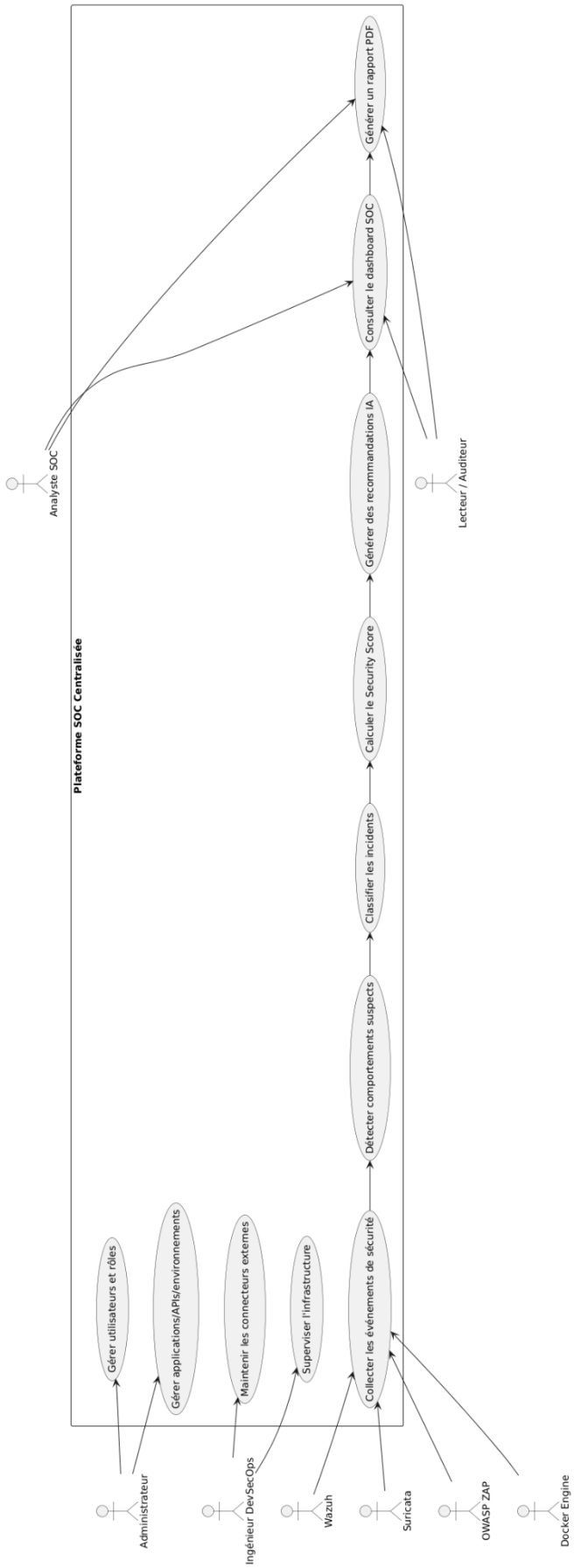


Figure 2.1: Global use-case diagram of CyberSOC

5.2 Use Cases by Module

The module diagrams detail the main responsibilities shown in the global view. Figures 2.2 and 2.3 concern application management and event collection. Figures 2.4 and 2.5 concern incident analysis and recommendations. Figures 2.6 and 2.7 show dashboard consultation and reporting.

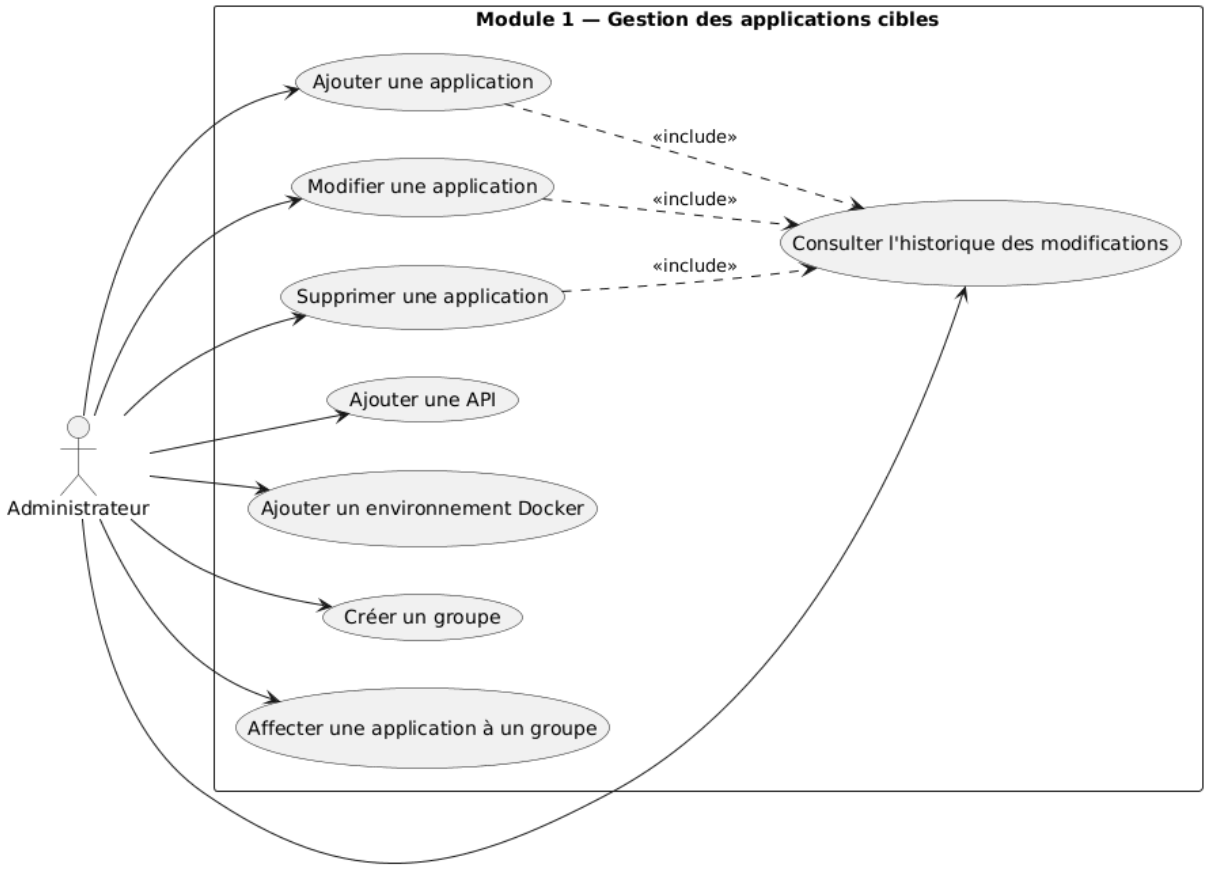


Figure 2.2: Use cases for managing monitored applications

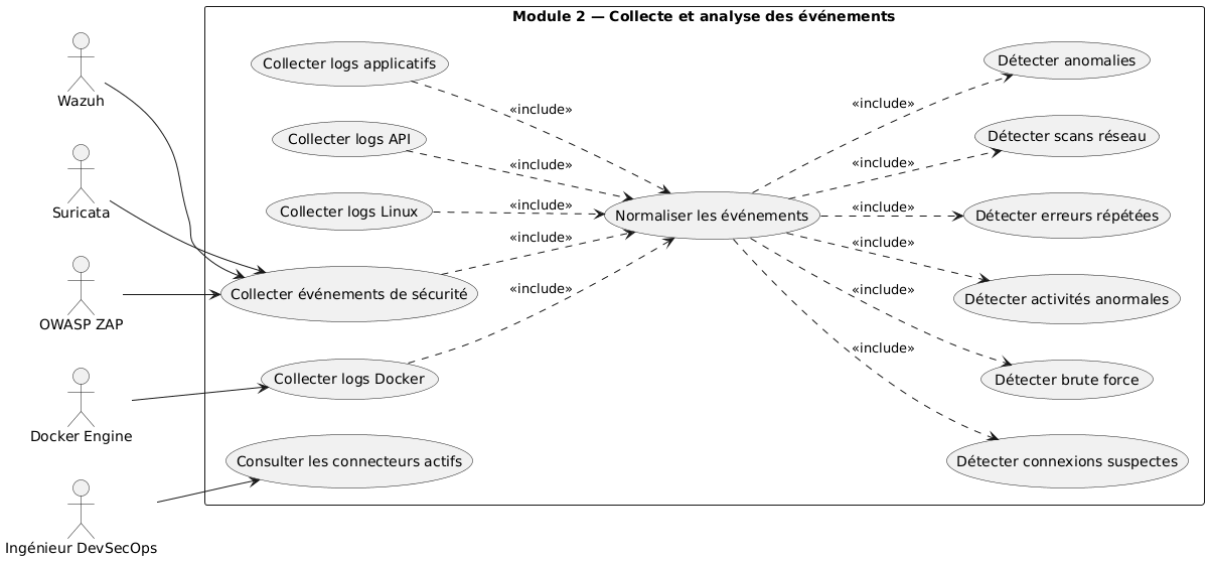


Figure 2.3: Use cases for collecting and analyzing events

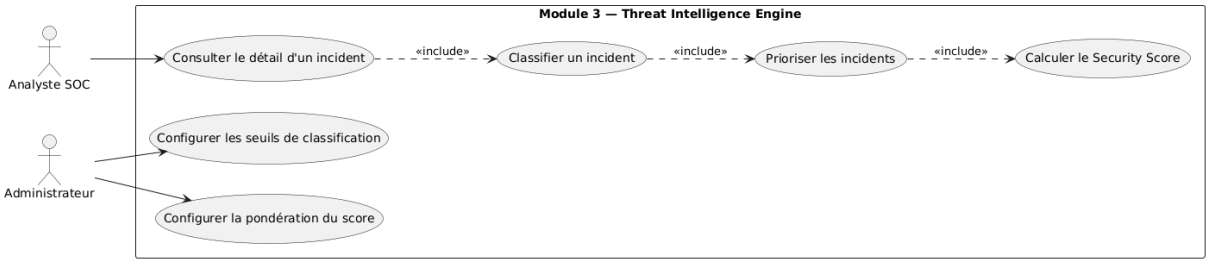


Figure 2.4: Use cases for incident analysis and security scoring

The recommendation diagram includes model evaluation and adjustment as a maintenance activity. The model used for event analysis is loaded for inference; it is not retrained for each incoming event. Figure 2.5 presents the recommendation module and its associated maintenance interaction.

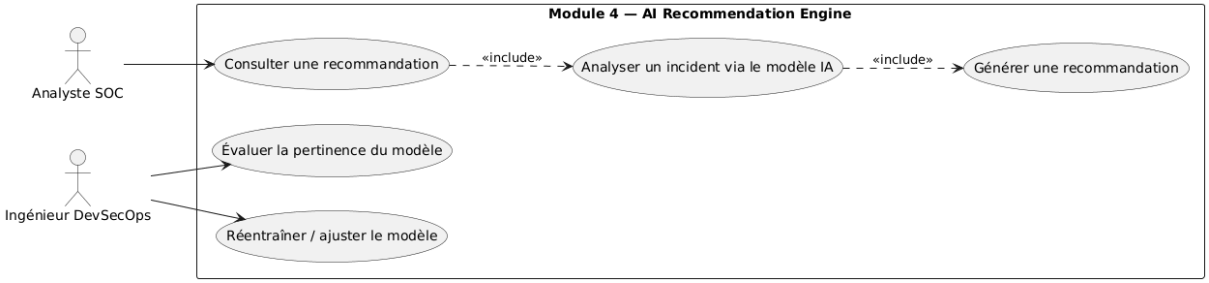


Figure 2.5: Use cases for recommendations and model maintenance

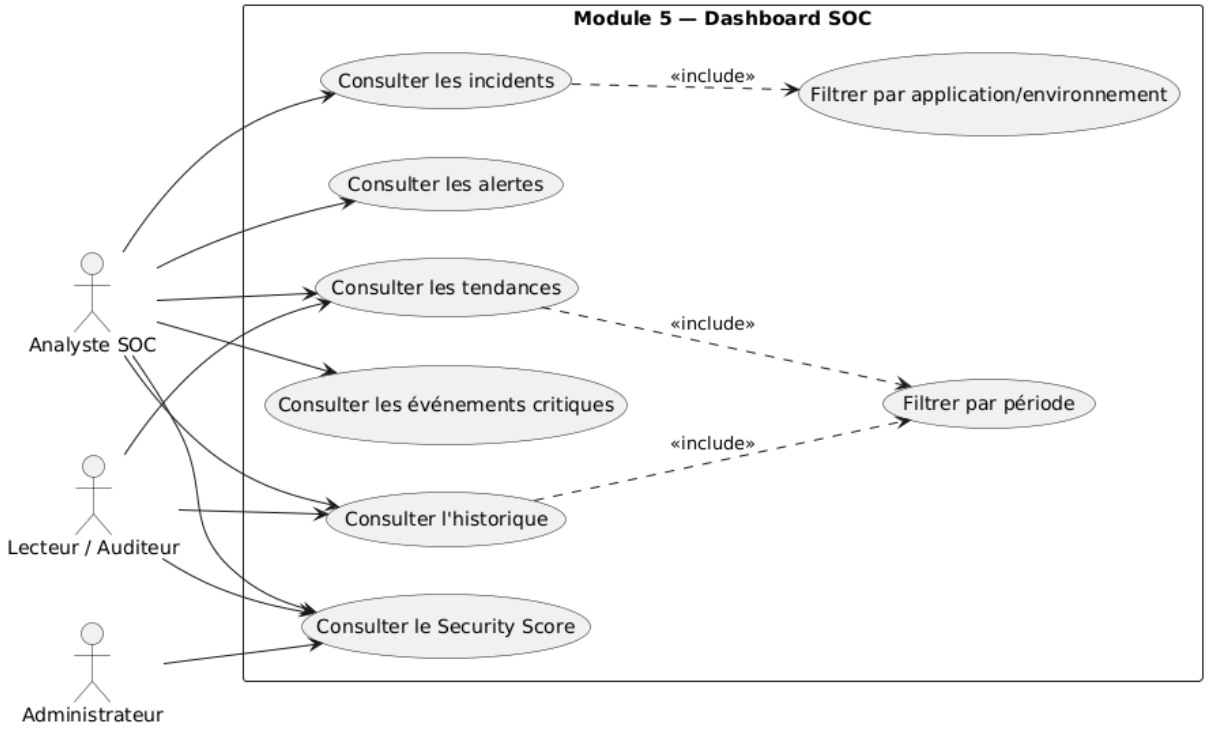


Figure 2.6: Use cases for the SOC dashboard

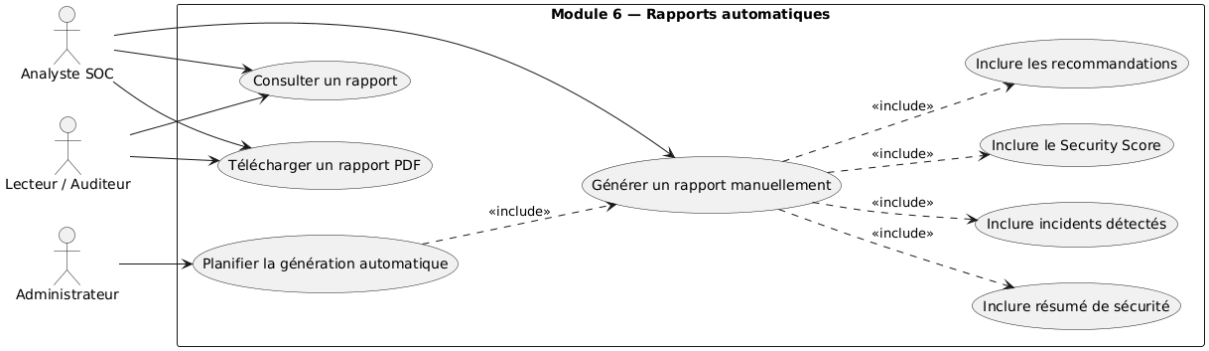


Figure 2.7: Use cases for security reports

Conclusion

This chapter identified the main users and external systems, summarized the key functional and quality requirements, and presented the project constraints. The use-case and sequence diagrams illustrated how the platform is expected to manage applications, process events, and provide reports. Chapter 3 now presents the architecture and technical design used to implement these requirements.

Chapter 3: Technology Study and Design

Introduction

Chapter 2 translated the project need into actors, functional requirements, non-functional requirements, and use cases. Building on that specification, this chapter explains the design choices made to implement the platform. It introduces the key security concepts, compares the selected technologies, and describes the architecture, event pipeline, data model, application mapping, and detection logic. It also gives the implemented Security Score calculation so that its meaning and limits are clear before Chapter 4 presents the software implementation.

1 Fundamental Concepts

SOC and Blue Team: A Security Operations Center (SOC) is the structure responsible for the continuous monitoring, detection, analysis and handling of security incidents within an organization. The Blue Team is its defensive component: it protects the information system, monitors its activity and reacts to incidents, as opposed to the Red Team, which simulates attacks.

SIEM and XDR: A Security Information and Event Management (SIEM) system centralizes the logs of several sources, correlates them according to rules and raises alerts. The Extended Detection and Response (XDR) approach extends this principle to the detection and response capabilities on the monitored hosts. Wazuh combines both functions and is used here as a source of host alerts [3].

Intrusion detection: An Intrusion Detection System (IDS) analyzes network traffic to find known attack signatures or unusual behavior. When it can also block traffic, it is called an Intrusion Prevention System (IPS). Suricata is used as a network detection tool [4].

Vulnerability scanning: Dynamic Application Security Testing (DAST) tests a running application by sending it requests to find flaws such as SQL injection or cross-site scripting. OWASP ZAP is used for this type of testing [5].

Log normalization: Each tool produces events in its own format, with its own field names and its own severity scale. Normalization converts these events into a common schema so that they can be analyzed uniformly.

Security Score: The Security Score is a synthetic indicator, between 0 and 100, that summarizes the security posture of an application from the incidents detected on it.

2 Technology Study

This section explains how we compared candidate technologies, summarizes the results, and gives the reasons for the tools selected for CyberSOC.

2.1 Evaluation Method

To compare the candidate technologies objectively, a common grid of six criteria was defined, each with a weight reflecting its importance for the project (Table 3.1). For each candidate, a mark from 1 (poor fit) to 5 (very good fit) is given on every criterion, and the final score is the weighted average:

$$S = \frac{\sum_i (m_i \times w_i)}{\sum_i w_i}, \qquad \sum_i w_i = 22.$$

Criterion	**Weight**	**Justification**
Functional coverage	5	Most decisive criterion: a tool that does not cover the functional requirements is disqualified whatever its other qualities
Ease of integration	4	Availability of an API or of an output format that the FastAPI backend can exploit easily
Deployment complexity	4	Ease of containerization and resources required, given the Docker constraint (RNF08) and the limited resources
Maturity and community	3	Documentation, project activity and community support
Cost and license	3	Open source, free and without blocking license constraint
Learning curve	3	Time needed to become operational within the duration of the project

Table 3.1: Evaluation criteria and weights

2.2 Comparison of Candidate Technologies

Table 3.2 summarizes the weighted scores obtained for each layer of the platform. The marks given on each criterion are detailed in the appendices.

Layer	Weighted scores	Retained
Host detection (SIEM)	Wazuh 4.05, Elastic Security 3.41, OSSEC 3.41	Wazuh
Network IDS/IPS	Suricata 3.82, Snort 3.77, Zeek 3.09	Suricata
Vulnerability scanning	Nuclei 4.18, OWASP ZAP 3.95, OpenVAS 3.73	OWASP ZAP
Database	PostgreSQL 4.68, MongoDB 3.91, TimescaleDB 3.68	PostgreSQL
Anomaly detection	Scikit-learn 4.45, expert rules only 4.41, PyOD 3.77	Expert rules and Scikit-learn
Backend	FastAPI 4.55, Express.js 4.27, Django REST 3.95	FastAPI
Frontend	Next.js 4.55, React SPA 4.45, Vue.js/Nuxt 4.14	Next.js
Containerization	Docker 4.68, Podman 3.86	Docker
Monitoring	Grafana 4.50, Prometheus alone 4.00, Kibana 3.64	Grafana with Prometheus

Table 3.2: Weighted scores of the candidate technologies

The three security tools deserve a closer look, since they provide the data on which the whole platform relies.

Wazuh. It covers natively the collection of security events (RF07), the detection of suspicious connections (RF08) and brute-force attempts (RF09), through predefined rules. The collection of application logs (RF06) and of repeated errors (RF11) is only partial and depends on the configuration of custom decoders and rules. Its REST API makes the alerts directly exploitable by the backend. Its main limit concerns network scans (RF12), which are not its primary function.

Suricata. It fills precisely this gap, since the detection of network scans is at the core of its operation. It analyzes the traffic in real time and produces alerts in EVE JSON format, which the collector can process directly. Its detection is based on signatures, so it only partially covers abnormal activities (RF10) and general anomalies (RF13).

OWASP ZAP. It does not collect logs but produces scan results that feed the Threat Intelligence Engine, and therefore contribute indirectly to the classification of incidents (RF14) and to the Security Score (RF16). It operates periodically, by scanning, and does not provide a continuous flow of events. Nuclei obtained a higher score thanks to its light integration, but ZAP was retained for the depth of its analysis of web applications and APIs and for its consistency with the initial specification. Nuclei may be added as a complement for fast scans of known vulnerabilities, and OpenVAS remains optional, depending on the available resources.

2.3 Justified Technology Choices

The choices below were validated by the supervisor. Table 3.3 gives the final stack with the reason for each choice.

Need	Tool	Main reason
Host detection and SIEM	Wazuh	Native coverage of RF07 to RF09 and REST API easy to integrate
Network detection	Suricata	Covers network scans (RF12) and produces structured EVE JSON output
Vulnerability scanning	OWASP ZAP	Deep DAST analysis of applications and APIs
Database	PostgreSQL	Strongly relational data model, with JSONB type for the raw events
Anomaly detection	Expert rules and Scikit-learn	Explainable rules for clear cases, model for diffuse anomalies
Backend	FastAPI	Automatic validation and documentation, same language (Python) as the machine learning module
Frontend	Next.js with Tailwind CSS	Structured organization suited to a dashboard that grows, fast styling
Infrastructure	Docker and Docker Compose	Reproducible, isolated and portable deployment, official images of the security tools
Monitoring	Grafana with Prometheus	Technical supervision distinct from the business dashboard; Prometheus supplies the metrics

Table 3.3: Technology stack retained

PostgreSQL was preferred to a NoSQL database because the data model is strongly relational, with precise cardinalities and integrity constraints, and the JSONB type makes it possible to store the raw data of the connectors without rigidity. TimescaleDB remains an option if the volume of events exceeds the capacity of a standard PostgreSQL instance. Prometheus was not mentioned in the initial specification, but it is required in practice to feed Grafana with infrastructure metrics.

3 Global Architecture

This section presents the platform from two complementary views. The high-level architecture shows its main components and how they exchange data. The deployment architecture then explains how these components are organized in Docker containers and networks.

3.1 High-Level Architecture

The architecture is organized in layers, as shown in Figure 3.1. The target applications are observed by the detection and collection layer, which comprises OWASP ZAP, Suricata and

the Wazuh Manager. Their output is retrieved by the orchestration layer, whose collector normalizes the events before handing them to the FastAPI backend. The backend relies on three modules: the Threat Intelligence Engine, which detects, classifies and scores, the AI Recommendation Engine and the report module. The results are stored in PostgreSQL and presented by the Next.js dashboard and by the PDF reports. In parallel, Prometheus and Grafana supervise the infrastructure.

Wazuh, Suricata, and Docker collection loops run as background tasks started by the FastAPI service. The SSH anomaly engine runs in a separate container. OWASP ZAP scans are started on demand through the API; they are not part of continuous polling.

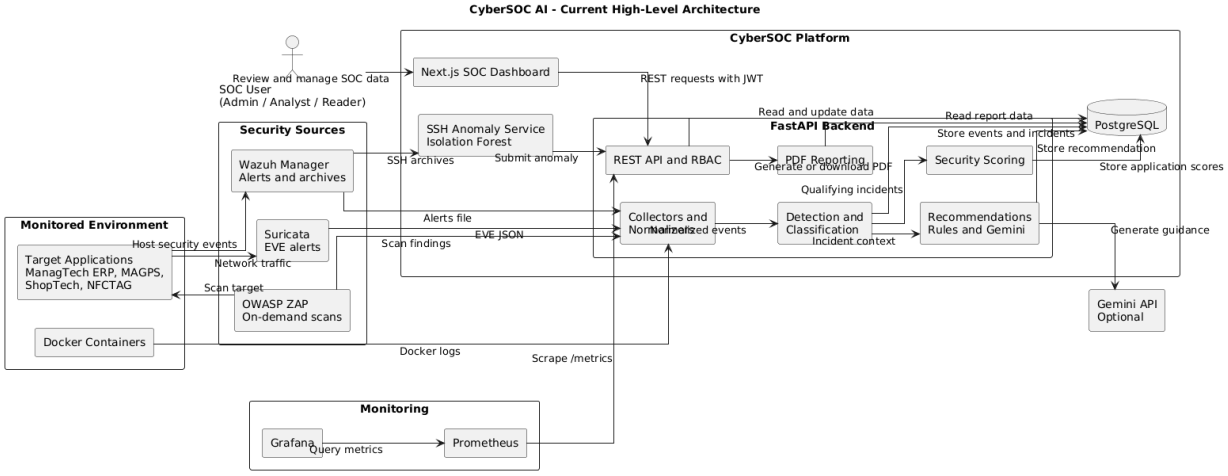


Figure 3.1: High-level architecture of the platform

3.2 Docker Deployment Architecture

The deployment topology in Figure 3.2 shows the actual Compose services and their networks. The backend, database, anomaly engine, monitoring services, and test application use the backend network. The frontend and security tools use their respective networks, while Suricata uses host networking. Collection tasks run inside the backend container; the separate `ml-engine` container handles SSH anomaly detection.

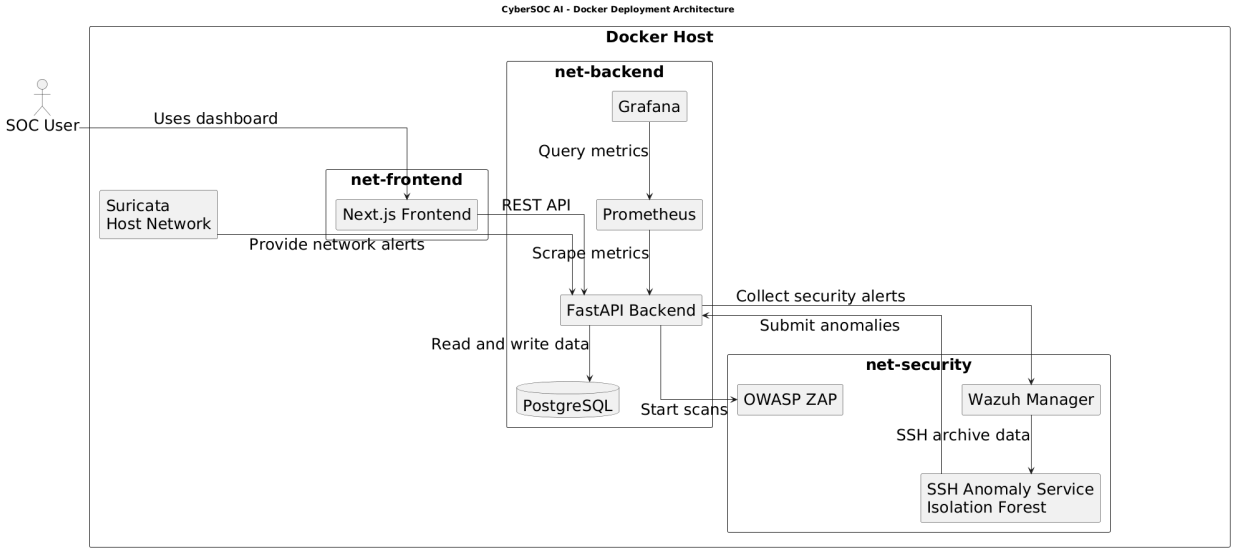


Figure 3.2: Docker deployment diagram

Three Docker networks are declared in the Compose file, as summarized in Table 3.4. The backend joins all three so it can communicate with the frontend, database, Wazuh, and ZAP. Suricata uses host networking and is therefore listed separately.

Network	Containers	Role
net-frontend	Next.js, FastAPI	Connects the dashboard to the backend API
net-backend	FastAPI, PostgreSQL, ml-engine, Prometheus, Grafana, Juice Shop	Application services, storage, monitoring, and the test application
net-security	FastAPI, Wazuh, OWASP ZAP	Security-tool communication; FastAPI requests Wazuh and ZAP services
Host network	Suricata	Gives Suricata access to the host network interface

Table 3.4: Docker networks and associated containers

Four named volumes preserve PostgreSQL, Wazuh, Prometheus, and Grafana data across container restarts. The anomaly engine also uses a bind-mounted state directory for its collector cursors.

4 Event Processing Design

This section explains how security data is prepared and processed by CyberSOC. It first compares the formats produced by the security tools, then presents the common event structure and the steps used to transform events into incidents.

4.1 Output Formats of the Security Tools

The three security tools produce data in JSON, but with different structures (Table 3.5). Three differences are structuring for the normalization: the naming of the fields, the severity scale, and the nature of the data, which is a real-time event for Wazuh and Suricata but the one-off result of a scan for ZAP.

Tool	Key fields	Severity scale
Wazuh	`rule.level`, `rule.description`, `agent.name`, `timestamp`, `full_log`	0 to 16, increasing
Suricata (EVE JSON)	`alert.signature`, `alert.severity`, `src_ip`, `dest_ip`, `proto`, `event_type`	1 to 3, where 1 is the most severe
OWASP ZAP	`alerts[].name`, `riskdesc`, `instances[].uri`, `cweid`	High, Medium, Low, Informational

Table 3.5: Output formats of the security tools

4.2 Normalized Event Schema

Two approaches were compared to normalize the events: a custom schema, designed for the needs of the project, and an alignment on the Elastic Common Schema (ECS), an open standard. The custom schema is simpler to design, to explain and to evolve, whereas ECS is recognized in the industry and would facilitate the integration with external tools, but it is richer and more complex than the current needs require. The supervisor validated the custom schema for the MVP. An alignment on ECS remains possible later if interoperability with external tools becomes necessary. The fields of the schema are listed in Table 3.6.

Field	**Description**
`event_id`	Unique identifier of the event (UUID)
`source`	Origin of the event: Wazuh, Suricata, ZAP or Docker
`source_raw_id`	Original identifier in the source tool, used to ignore duplicates
`timestamp`	Date and time of the event, in ISO 8601
`application_id`	Reference to the target application or environment
`event_type`	Type of event: suspicious connection, brute force, network scan, vulnerability, anomaly
`severity_normalized`	Normalized level: Critical, High, Medium or Low
`severity_raw`	Original value (for example `rule.level`=10, `severity`=1 or High)
`description`	Descriptive text of the event
`network_context`	Source address, destination address and protocol
`raw_data`	Original data, kept for audit and debugging

Table 3.6: Fields of the normalized event schema

4.3 Severity Conversion

The normalized severity is obtained through a conversion table specific to each source (Table 3.7). It reconciles the three scales, including the inverted scale of Suricata.

Normalized level	**Wazuh** (`rule.level`)	**Suricata** (`alert.severity`)	**ZAP** (`riskdesc`)
Critical	12 to 16	1	High
High	8 to 11	2	Medium
Medium	4 to 7	3	Low
Low	0 to 3	none	Informational

Table 3.7: Conversion of the severity levels by source

4.4 Processing Pipeline

The processing of an event follows the activity diagram of Figure 3.3. A raw event received from a source is normalized and stored in the database. The detection then determines whether a suspicious pattern is present. If it is not, the event is archived without creating an incident. If it is, an incident is created and classified (Critical, High, Medium or Low), the Security Score is calculated or updated, a recommendation is generated, and the incident, the score and the recommendation are stored. The dashboard is then notified and, when the incident is Critical, a priority alert is triggered.

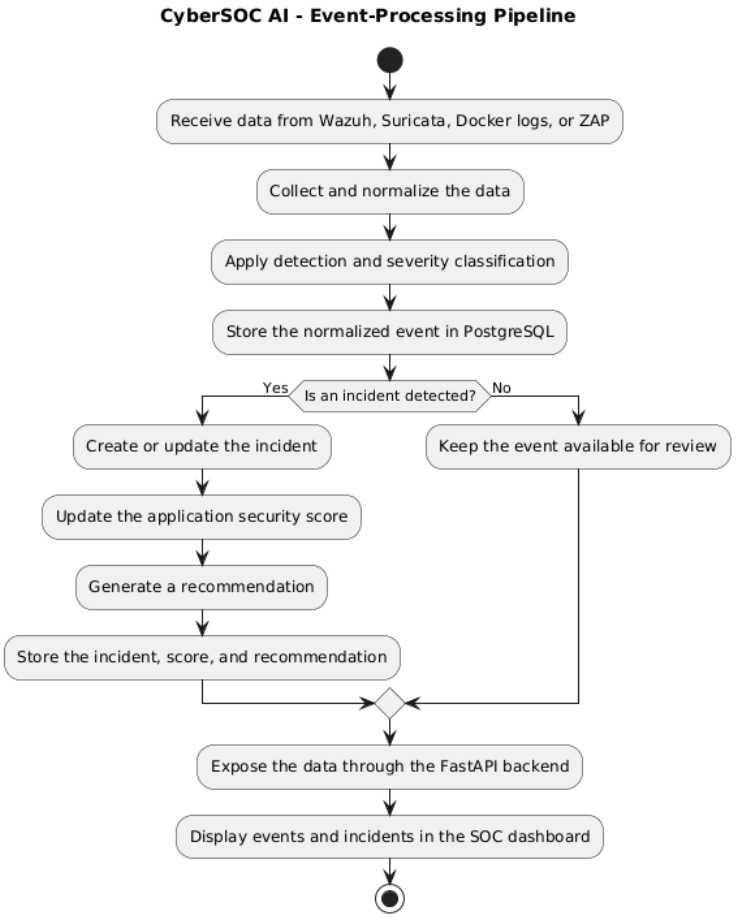


Figure 3.3: Activity diagram of the event processing pipeline

The pipeline implemented in the MVP contains eight steps: collection from Wazuh, normalization, storage, detection, incident creation, Security Score, dashboard and PDF report. The other sources were plugged into the same chain in the following iteration.

5 Class Diagram

This section presents the main data entities used by CyberSOC and their relationships. The simplified class diagram is shown here for readability; a more detailed editable PlantUML model is included in the annex.

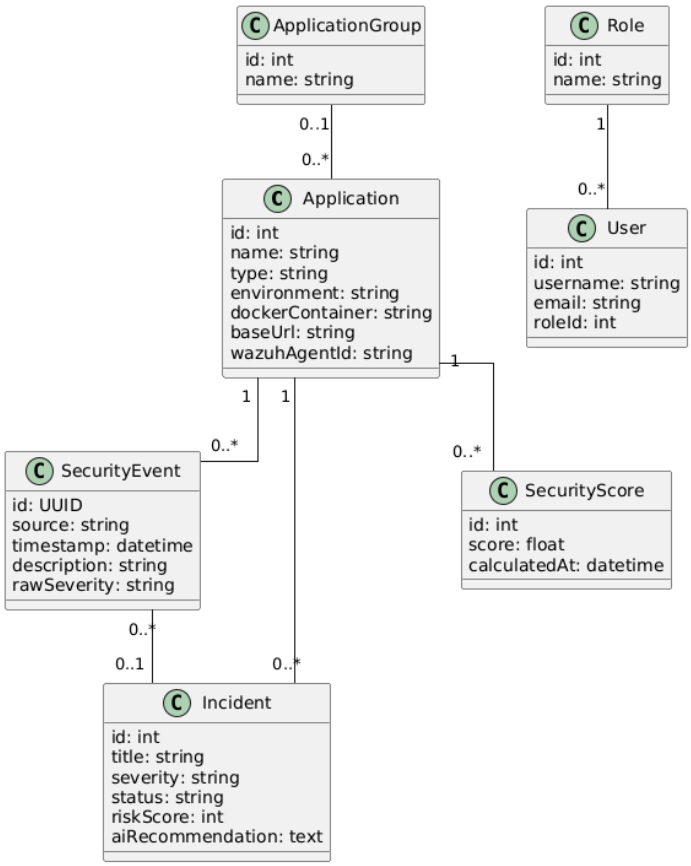


Figure 3.4: Class diagram of the platform

6 Application Registration and Source Mapping

The application model stores basic information about each target and optional fields for connecting it to a data source. The main fields and their purposes are:

- `type` and `environnement` describe the application category and environment.
- `docker_container` lets the Docker resolver match logs to a registered container. If no match is found, it uses the default application ID.
- Wazuh agents are linked through the `ApplicationSourceMapping` table, which stores the source, resource type, resource value, and whether the mapping is active. The code can create an application and mapping when it first encounters a new agent.
- `base_url` stores a web application's address. It can be supplied with the application ID to start a ZAP scan; adding the URL alone does not start monitoring.
- `hostname` and `ip_address` store extra target information. They do not, by themselves, connect the application to incoming events.

The Docker and Wazuh paths were tested during the project. The end-to-end URL scenario was not completed, so it is not presented as a validated integration.

7 Main Scenarios

This section presents the main ways users and security tools interact with CyberSOC. We first summarize four scenarios, then illustrate three of them with sequence diagrams.

7.1 Scenario Summaries

The following scenarios cover event processing, report generation, application registration, and on-demand security scanning.

1. **Brute-force alert processing:** Wazuh alerts are imported and linked to an application through the Wazuh agent mapping. The events are normalized and analyzed. When a brute-force incident is detected, it is stored and linked to its event, and the application's Security Score is recalculated.

2. **PDF report generation:** An authorized user requests a report by selecting an application and a period. The backend retrieves the relevant incidents and Security Score, generates the PDF, and returns it for download.

3. **Adding a target application:** The administrator submits the application details through the interface. The backend checks whether an application with the same name already exists. If the name is available, the application is saved and returned. Otherwise, the backend returns an error.

4. **ZAP scan and alert import:** An authorized user submits a target URL and application ID to start a scan. The backend runs the ZAP Spider and Active Scan and waits for them to finish. It then imports the resulting alerts as events. These events pass through the shared processing pipeline and may create incidents. The application's Security Score is recalculated when events are imported.

The next subsection presents sequence diagrams for the first three scenarios. The ZAP scan is summarized above but is not represented by a sequence diagram here.

7.2 Sequence Diagrams

Each diagram shows the messages exchanged between the user, the platform, and the relevant service during a scenario. Figure 3.5 follows a Wazuh alert through analysis. Figure 3.6 illustrates report generation, and Figure 3.7 shows application registration.

Figure 3.5 shows how a Wazuh brute-force alert is received and processed by the platform.

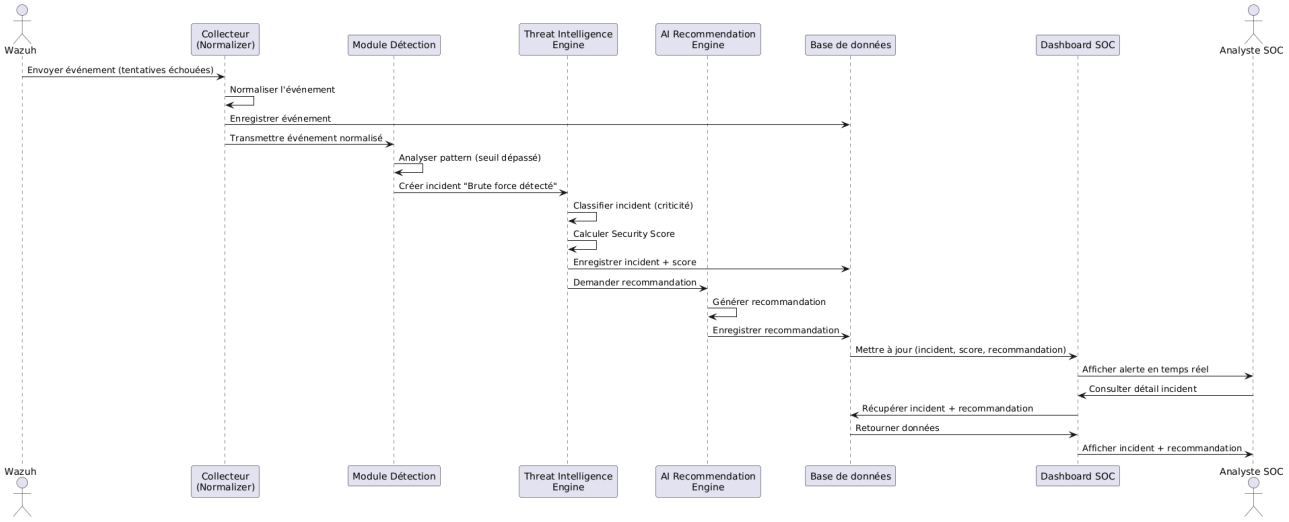


Figure 3.5: Sequence diagram for analyzing a brute-force alert

The diagram shows the alert moving from Wazuh to the backend, where it is normalized and checked. When the event matches a detection rule, CyberSOC records an incident, updates the application's score, and makes the result available on the dashboard.

Figure 3.6 presents the request and response flow for generating a PDF report.

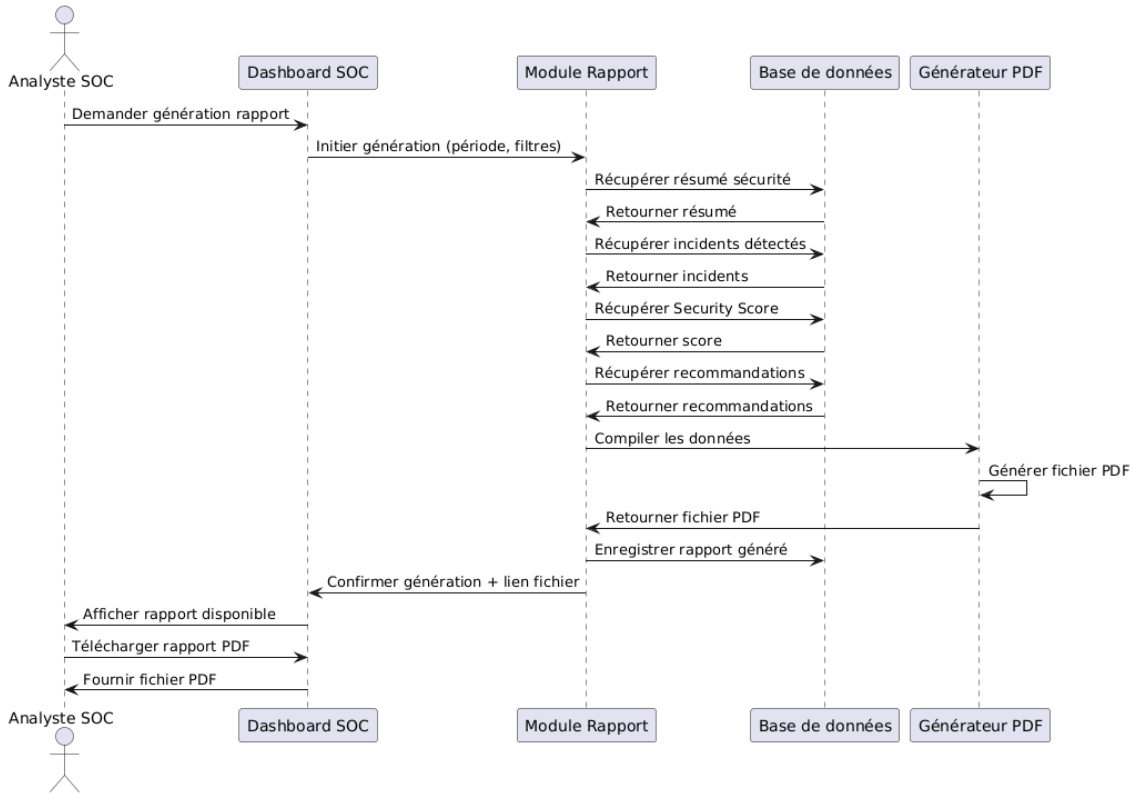


Figure 3.6: Sequence diagram for generating a PDF report

The sequence shows an authorized user selecting an application and period. The backend gathers the corresponding security information, generates a PDF, and returns the file for download.

Figure 3.7 shows how an administrator submits an application and how the backend validates and saves it.

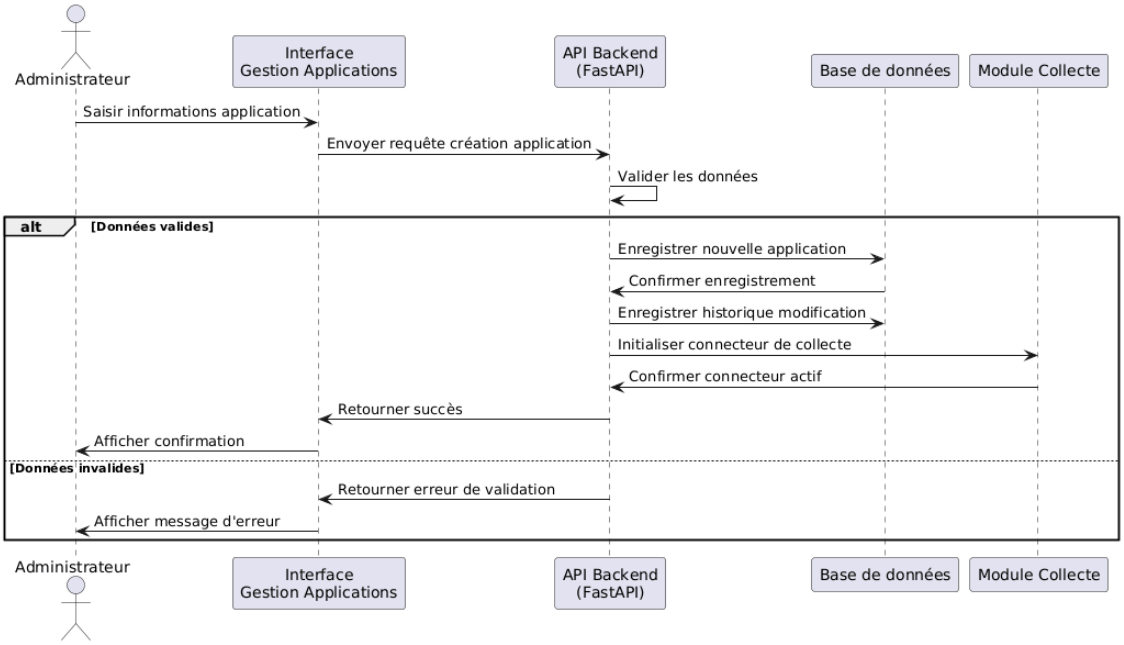


Figure 3.7: Sequence diagram for adding a monitored application

The administrator submits the application details, the backend checks the data and name, and the database stores valid information. If validation fails or the name already exists, the backend returns an error instead of creating the record. These scenarios connect the requirements to the platform's expected interactions; the next chapter explains how we implemented them.

8 Detection, Scoring and Recommendation Design

This section presents the design of incident detection, severity classification, application scoring, recommendations, and anomaly detection.

1. **Detection and Classification:** CyberSOC analyzes normalized events to identify suspicious activity, such as brute-force attempts, suspicious connections, network scans, and repeated errors. When an event meets the detection criteria, the platform creates an incident and assigns it a severity level: Critical, High, Medium, or Low. This helps users identify which incidents need attention first.

2. **Security Score:** The Security Score summarizes an application's security status on a scale from 0 to 100. A higher score represents a healthier status. It considers the severity and status of incidents associated with the application, helping users compare applications and follow changes over time.

3. **Recommendations:** For each incident, CyberSOC provides guidance based on its type and severity. These recommendations help users understand possible response measures and are presented with incident details and reports.

4. **Anomaly Detection:** The design combines detection rules with machine learning. Rules address recognizable cases, such as brute-force attempts. An Isolation Forest model looks for less obvious patterns by analyzing events grouped by source address over five-minute periods [7]. The model is trained offline and is not retrained for each incoming event.

9 Security Design

This section presents the planned approach to user authentication and access control. CyberSOC defines three roles: Administrator, SOC Analyst, and Reader. Table 3.8 summarizes their access to the main features, including incident updates, reporting, and on-demand scans.

1. **Authentication:** Users access the platform with an email address and password. After successful authentication, they receive a token to access protected features. Passwords must be stored securely, and access is refused when authentication is missing or invalid.

2. **Role-Based Access Control:** Each role has permissions suited to its responsibilities. Access to a protected feature is limited to the roles authorized for that action.

3. **Secrets and Network Isolation:** Sensitive configuration values are kept separate from the source code. The platform's services are separated into network groups to limit unnecessary communication between components and help protect services such as the database.

The Administrator manages users and monitored applications. The SOC Analyst investigates incidents, updates their status, generates reports, and launches scans. The Reader consults security information in read-only mode.

Table 3.8: Planned access rights by role

Feature	**Administrator**	**SOC Analyst**	**Reader**
Users and roles	Manage	No access	No access
Applications and groups	Manage	Read	Read
Dashboard and scores	Read	Read	Read
Incidents	Read	Read and update status	Read
Recommendations	Read	Read	Read
Reports	Generate and download	Generate and download	View available reports
OWASP ZAP scans	Launch and view results	Launch and view results	View results
Security connectors and settings	Manage	View	View

Conclusion

This chapter compared technologies, selected the project stack, and presented the platform design, including its architecture, data model, detection, scoring, recommendations, and security. The next chapter explains how we implemented this design.

Chapter 4: Implementation

Introduction

The previous chapter described how the platform was designed. This chapter describes how it was built. It presents the development environment and the technology stack, the installation and deployment of the platform, then the work carried out in each phase of the project. The resulting application is then shown module by module, through screenshots. The chapter ends with the technical difficulties encountered, the solutions adopted and the current limits of the platform.

1 Development Environment

The platform was developed on local workstations and run as a set of Docker Compose services. We used a web browser to access the dashboard and the automatically generated Swagger documentation for the FastAPI endpoints. Docker Compose made it possible to start the backend, frontend, database, Wazuh manager, machine-learning service, and supporting monitoring and security tools as a coordinated environment. The commands and test procedures used to verify this environment are detailed in Chapter 5.

Source code management relies on Git and the branch workflow introduced in Chapter 1. The initial plan guided integration, while the order of tasks and their timing were adapted as dependencies and test results became clear. The code, including variable names, functions, and comments, is written in English.

2 Technology Stack

The tools selected during the design phase were used to build the CyberSOC platform. The following list groups them by their role and describes how each one was used in the project.

2.1 Application Development

- **Python:** We used Python to develop the backend services for processing events, classifying incidents, calculating security scores, and generating recommendations.
- **FastAPI:** We used FastAPI to build the backend API that connects the dashboard with the platform's security and management features.
- **Next.js:** We used Next.js to build the web dashboard for viewing applications, events, incidents, and scores.
- **Tailwind CSS:** We used Tailwind CSS to style the dashboard pages and their interface components.

2.2 Data Storage and Deployment

- **PostgreSQL:** We used PostgreSQL to store the platform's application, event, incident, score, and user data.
- **Docker and Docker Compose:** We used Docker Compose to run the platform services as containers and configure their networks and shared data [6].

2.3 Security Monitoring and Analysis

- **Wazuh:** We used Wazuh agents to collect host security alerts and send them to CyberSOC for processing.
- **Suricata:** We integrated Suricata network events into the platform's security event processing.
- **OWASP ZAP:** We integrated on-demand Spider and Active scans. Their alerts are imported into CyberSOC and processed as security events.
- **Scikit-learn:** We used its Isolation Forest model in the separate `ml-engine` container to identify unusual SSH activity. The model is trained offline and does not retrain for each event.

2.4 Metrics and Visualization

- **Prometheus:** We configured Prometheus to collect metrics exposed by the backend.
- **Grafana:** We used Grafana dashboards to display metrics collected by Prometheus.

The logos identify the tools used in the project. Their individual source links are listed in the image credits after the bibliography.

3 Installation, Configuration and Deployment

This section describes the project structure, the services defined in Docker Compose, the configuration files, and the checks recorded during setup.

3.1 Repository Structure

The main project directories and files are:

1. `backend`: contains the FastAPI application.
2. `frontend`: contains the CyberSOC dashboard pages.
3. `infra`: contains the Docker Compose file, database initialization script, and security-tool configuration.

4. `ml-engine`: contains the SSH anomaly detection service and its model.
5. `grafana-dashboards`: contains the Grafana dashboard configuration.
6. `.env.example`: provides a template for the project configuration variables.

3.2 Docker Compose Setup

The `infra/docker-compose.yml` file defines the services shown in Table 4.1. The backend connects to the frontend, backend, and security networks. Suricata uses host networking to access the host network interface. The other services use the declared Docker networks.

Table 4.1 lists each service, its purpose, and its published host port.

Table 4.1: Services defined in the Docker Compose file

Service	**Purpose**	**Host port**
PostgreSQL	Stores platform data	5432
Backend	FastAPI application and API	8000
ML Engine	Processes SSH activity for anomaly detection	Not published
Frontend	CyberSOC dashboard	3000
Wazuh	Host monitoring and alert collection	15140, 15150, 55005
Suricata	Network monitoring; uses host networking	Host network
ZAP	On-demand web application scans	8081
Juice Shop	Test web application	3002
Prometheus	Collects application metrics	9090
Grafana	Displays monitoring dashboards	3001

The Compose file declares three Docker networks: `net-frontend`, `net-backend`, and `net-security`. It also defines four named volumes for PostgreSQL, Wazuh, Prometheus, and Grafana data. Additional bind mounts provide the backend with security logs and configuration files.

3.3 Environment Variables and Secrets

The `.env.example` file lists configuration values for PostgreSQL, the JWT, the backend, the frontend, Wazuh, the machine-learning service, and Gemini. It is a template; the values used to run the platform are supplied through the local environment and Docker Compose configuration.

The current Compose file also defines Grafana's initial administrator credentials directly in the configuration. Therefore, the current project configuration does not keep every credential in a separate local environment file.

3.4 Database Initialization

The `infra/init.sql` script defines 20 database tables for users, roles, groups, applications, events, incidents, scores, recommendations, reports, schedules, and related records. Docker mounts this script into PostgreSQL's initialization directory. PostgreSQL runs it when it initializes a new, empty data directory. The backend also calls SQLAlchemy's `create_all` during startup to create tables represented by its models.

3.5 Setup Verification

The project files and captured backend logs provide the setup evidence summarized in Table 4.2. The table distinguishes configured services from checks visible in the logs.

Table 4.2: Setup checks and available evidence

Check	**Evidence**
Backend health route returns `{"status":"ok"}`	Route defined in the backend
Backend metrics endpoint responds with HTTP 200	Recorded in backend logs
Wazuh, Suricata, and Docker polling loops start	Recorded in backend logs
Security alert files are available to the backend	A log warning reports that an alerts file was not found

The captured logs confirm backend startup, metric requests, and the start of the polling loops. They also include an alert-file warning, so this setup evidence alone does not confirm that every source was successfully imported during that run.

4 Implementation by Phase

The implementation was organized into three phases. Each phase had a specific objective, and its tasks were checked using the tests and evidence presented in Chapter 5.

4.1 Phase 2: MVP Core

The objective of Phase 2 was to create a working MVP that processes Wazuh alerts and presents the resulting security information to users. Table 4.3 summarizes the main tasks and their validation.

Table 4.3: Phase 2 tasks and validation

Task	Validation
Docker environment and FastAPI backend	Startup, health endpoint, and Swagger checked
Authentication and the three user roles	Registration, login, user information, and access errors tested
Application management	Create, read, update, and delete operations tested
Wazuh alert collection and event normalization	Agent-based import and event storage tested
Incident detection, classification, and Security Score	Tested with security events and recorded incidents
SOC dashboard and manual PDF report	Dashboard data and PDF generation checked

4.2 Phase 3: Security Integrations

The objective of Phase 3 was to add security sources and complete the main management and investigation workflows. Table 4.4 summarizes the tasks and the available validation evidence.

Table 4.4: Phase 3 tasks and validation

Task	Validation
Suricata and Docker log collection	Collector behavior tested with source records
Wazuh Classification v2	Risk score and severity mapping applied to rule details and repeated alerts
On-demand OWASP ZAP scans and alert import	Scan completed; 7 alerts received, 6 events imported, and 3 incidents created
Groups, user administration, incident details, and filters	Main interface workflows checked; some application-related paths remain untested
Role-based interface and access control	Administrator, SOC Analyst, and Reader workflows tested
Kali Linux and Ubuntu security scenario	Wazuh agent events were received and reviewed in CyberSOC
Application URL mapping	End-to-end validation not completed

4.3 Phase 4: AI, Monitoring and Advanced Reporting

The objective of Phase 4 was to add SSH anomaly detection, monitoring, and advanced platform capabilities. Table 4.5 summarizes the tasks and their validation status.

Table 4.5: Phase 4 tasks and validation

Task	Validation
Offline training and use of the SSH Isolation Forest model	Normal and anomalous SSH behavior tested
Machine-learning incident creation and duplicate handling	Creation, duplicate detection, and enrichment scenarios tested
Suricata and Docker anomaly models	Source collection implemented; dedicated models remain outside the validated scope
Template-based recommendations and optional Gemini integration	Template recommendations checked; Gemini remains optional
Prometheus metrics collection and Grafana configuration	Backend metrics endpoint returned HTTP 200; Grafana configuration is present
Scheduled reports	Not included in the validated test evidence

4.4 Classification and Security Score

For Wazuh events, Classification v2 calculates a risk value from the rule level and context. It adds one point when a sensitive rule group is present and two points for a high-risk rule ID. Repeated alerts add one point for 3–5 occurrences, two points for 6–10, and three points for more than 10. Excluded groups receive a Low severity and a risk value of zero. The resulting risk value is mapped to a severity level in Table 4.6.

Table 4.6 shows how the Wazuh risk value maps to the four incident severity levels.

Table 4.6: Wazuh risk value and severity mapping

Risk value	Severity
12 or higher	Critical
8–11	High
4–7	Medium
Below 4	Low

The application Security Score uses open or in-progress incidents from the last 30 days. Incidents are grouped by source and normalized title; up to the three highest risk values in each group contribute to the penalty. Negative risk values count as zero. If an incident has no risk value, the fallback values are 12 for Critical, 9 for High, 6 for Medium, and 3 for Low. Resolved or older incidents are excluded. The final score is:

$$\text{Score} = \max\big(0, \min(100, 100 - \text{total penalty})\big).$$

For example, a total penalty of 36 gives a score of 64. The score is a rule-based indicator of current incident load, not a probability of attack.

5 Presentation of the Platform

This section presents selected CyberSOC screens, with one representative image for each main workflow. Additional screenshots are provided in the appendix.

5.1 Dashboard

Users sign in and access a dashboard suited to their role. Figure 4.1 shows the administrator dashboard, including the main security indicators and recent incidents.

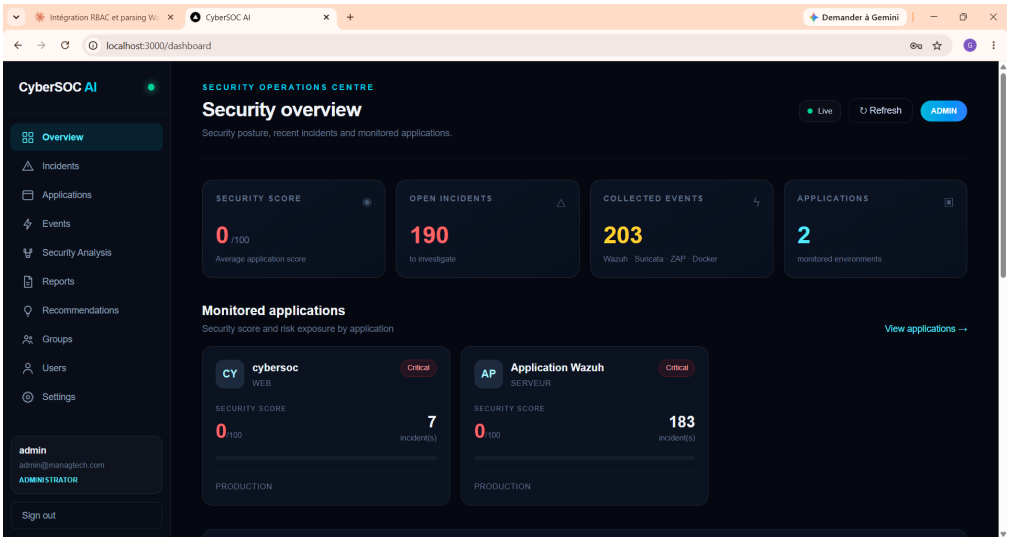


Figure 4.1: Administrator dashboard

5.2 Incident Management

The incident page helps users review detected incidents and their severity, status, and source. Figure 4.2 shows the incident list.

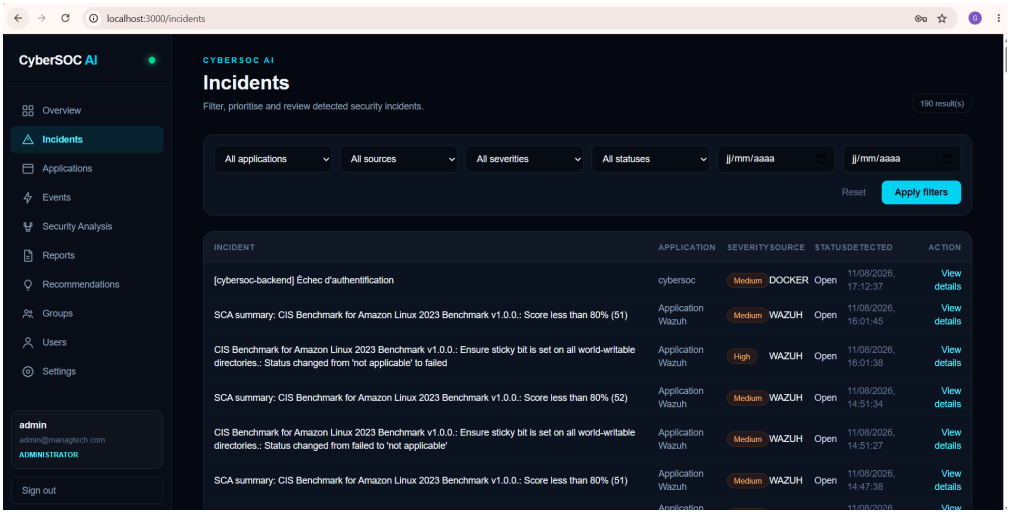


Figure 4.2: Incident list in CyberSOC

5.3 Administration

Administrators can manage user accounts and assign roles. They can also organize applications into groups. Figure 4.3 shows the user management page.

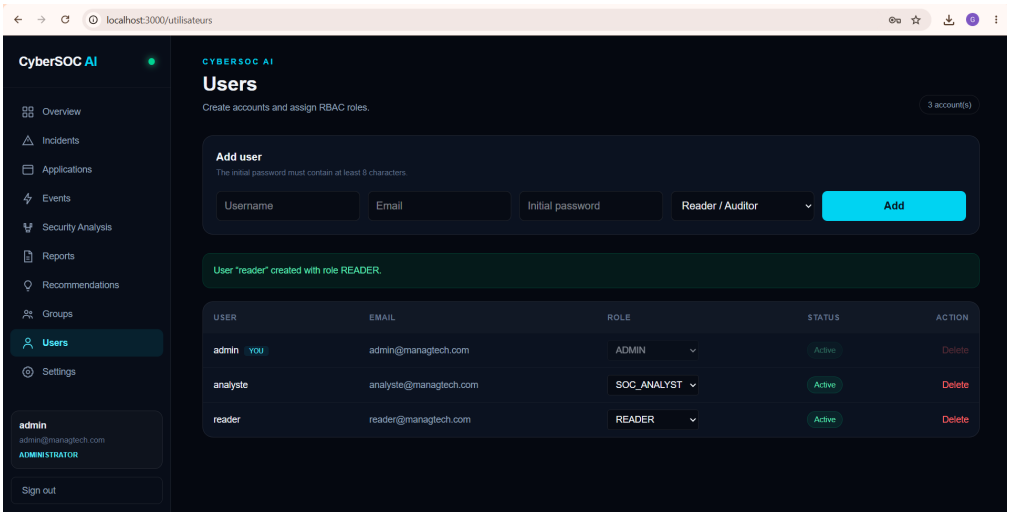


Figure 4.3: User management page

5.4 Security Analysis with ZAP

Administrators and SOC Analysts can start an OWASP ZAP scan for a target application. Figure 4.4 shows the findings returned to CyberSOC after a scan.

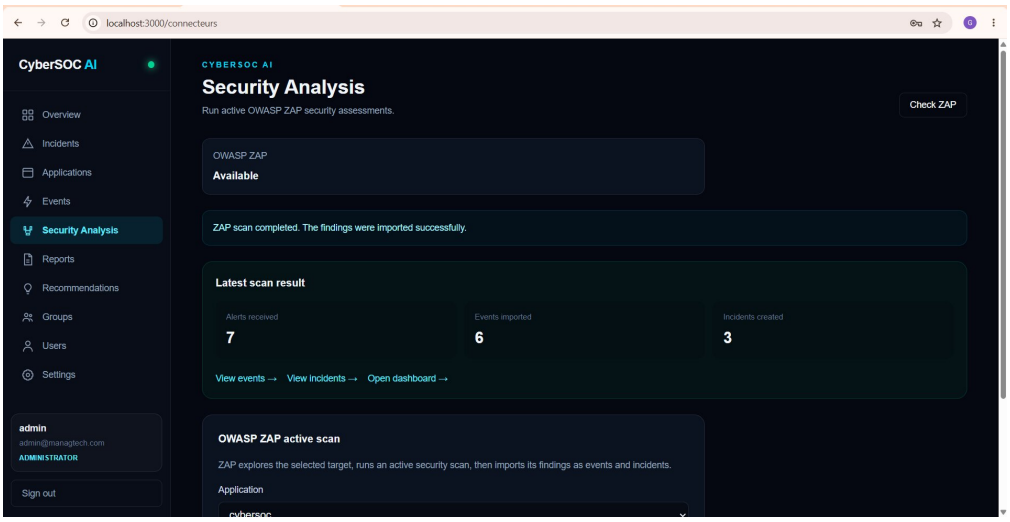


Figure 4.4: OWASP ZAP scan results

5.5 Reports

Authorized users can generate PDF reports for an application and period. Figure 4.5 shows the reports page.

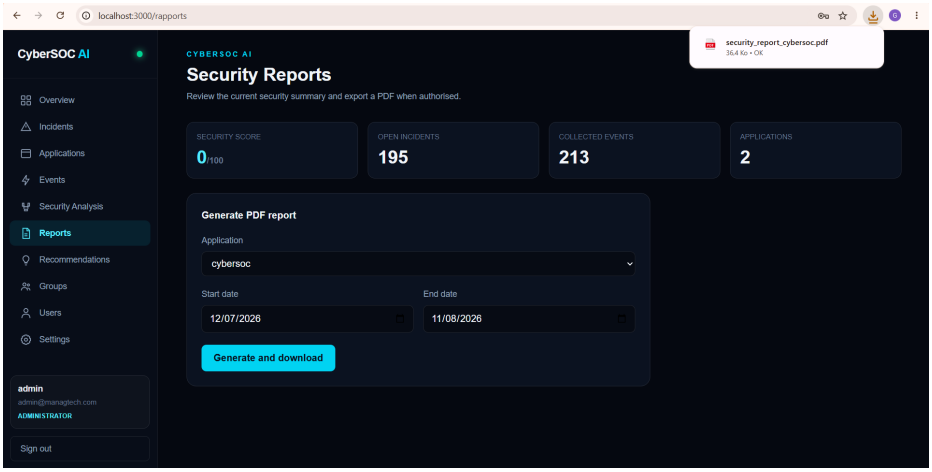


Figure 4.5: PDF reports page

6 Technical Challenges and Solutions

Connectivity between the backend and Wazuh. The backend could not reach the Wazuh API at first, since the two services were not on the same Docker network. Attaching the backend to the security network solved the problem, and the verification from inside the container confirmed it.

Duplicate alerts. Since the alerts are retrieved by regular polling, the same alert could be imported several times. The unique constraint on the raw identifier of the events makes the import idempotent, and a second import returns zero new event.

Heterogeneous severity scales. Wazuh, Suricata and ZAP do not use the same scale, and Suricata uses an inverted one. A conversion table by source brings them back to the four normalized levels while the original value is preserved.

On-demand ZAP scans. A scan is started by a user through the FastAPI endpoint. The endpoint runs the Spider and Active Scan, checks their progress, imports the findings, and returns the result when processing is complete. This is a request-triggered workflow, not continuous collection or a separate background job.

Test environment. Validation was carried out in the project environment with Wazuh agents and replayed security logs. It was not a long-running production deployment, so production availability and load are outside the evidence collected.

Calculation of the Security Score. In the first version of the MVP, the score was computed from a list of incidents sent in the request, which made it possible to validate the logic of the calculation without depending on the persistence. This solution was replaced by the retrieval of the incidents of the application from the database, once their persistence was integrated.

Team integration. The first phase was slowed down by the formation of the team and the learning of the tools. Shared technical discussions and integration checks helped the team harmonize its practices and resolve dependencies as the platform developed.

Conclusion

This chapter described the implementation of the platform: an environment deployed with Docker Compose, a pipeline built phase after phase, from the collection of Wazuh alerts to the PDF report, and the integration of the other sources, of the ZAP scans and of the management features. The application was presented through its main screens for the three roles, and the difficulties encountered and the remaining limits were discussed. The next chapter presents the tests carried out to validate the functionalities, the pipeline and the access control.

Chapter 5: Testing and Validation

Introduction

Chapter 4 described how the CyberSOC platform was implemented and presented its main pages and services. This chapter now explains how we checked those functions. It details the environment and commands, Swagger and database checks, dashboard and role tests, source integrations, attack and scan scenarios, and the machine-learning checks. It then summarizes the results and states which parts, including URL-based testing, still need validation.

1 Validation Approach

Validation was carried out in the project environment through the following checks:

- We checked that the services started and that the API routes returned expected responses.
- We checked that events and incidents were stored in PostgreSQL.
- We confirmed that the dashboard displayed the stored data.
- We ran integration scenarios covering collection, normalization, detection, scoring, and presentation.
- We used Wazuh agents and replayed anonymized authentication and Fail2ban logs to provide test data.
- We did not complete end-to-end testing of application URLs. The tests therefore cover the documented scenarios, not production-scale performance or every possible attack.

2 Functional Test Scenarios

We tested the main tasks from each implementation phase. The checks covered service startup, application management, authentication and roles, security-source integrations, incident detection, scoring, recommendations, reports, and machine-learning scenarios. Figures 5.1 and 5.2 show selected test evidence; additional screenshots are provided in the annex.

Figure 5.1 shows the Wazuh alert import used to validate the Phase 2 collection workflow. The service startup and Swagger checks were also completed; additional screenshots are provided in the annex.

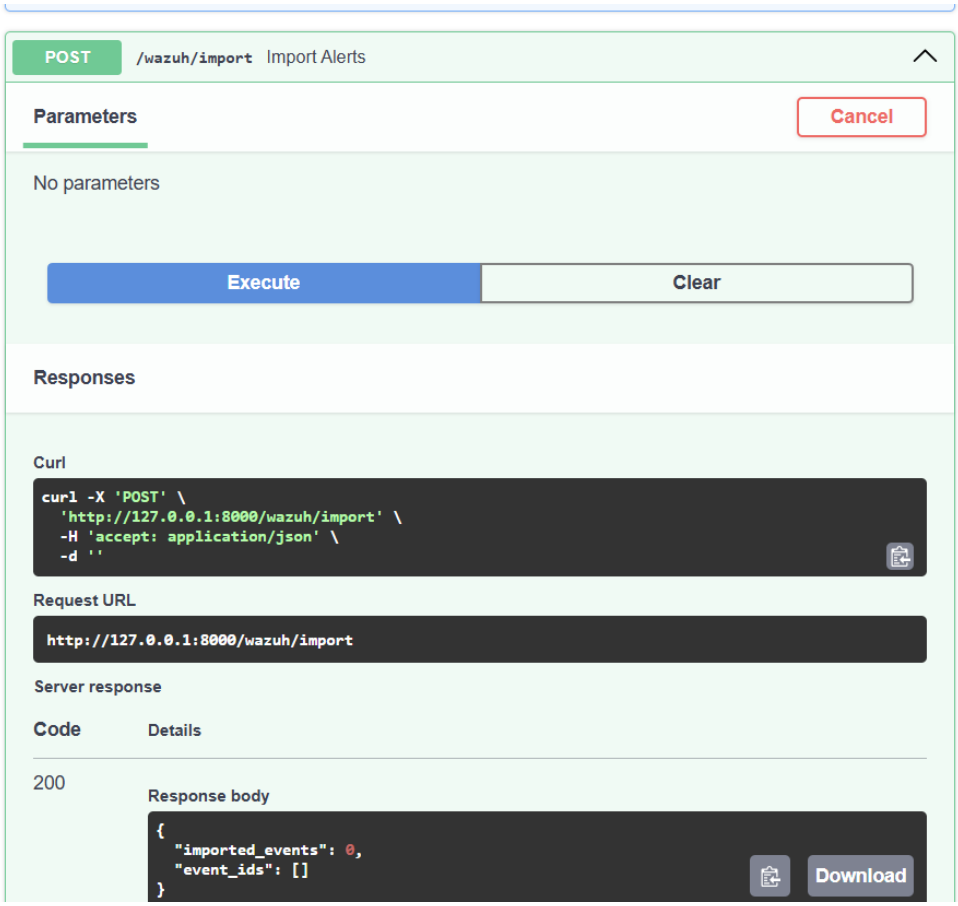


Figure 5.1: Wazuh alert import during Phase 2 testing

For Phase 3, we tested the additional integrations, replayed security logs, and role-specific access. The ZAP scan completed with 7 alerts received, 6 events imported, and 3 incidents created. The URL-based application-mapping path was not tested end to end.

Figure 5.2 shows the ZAP scan and its imported findings.

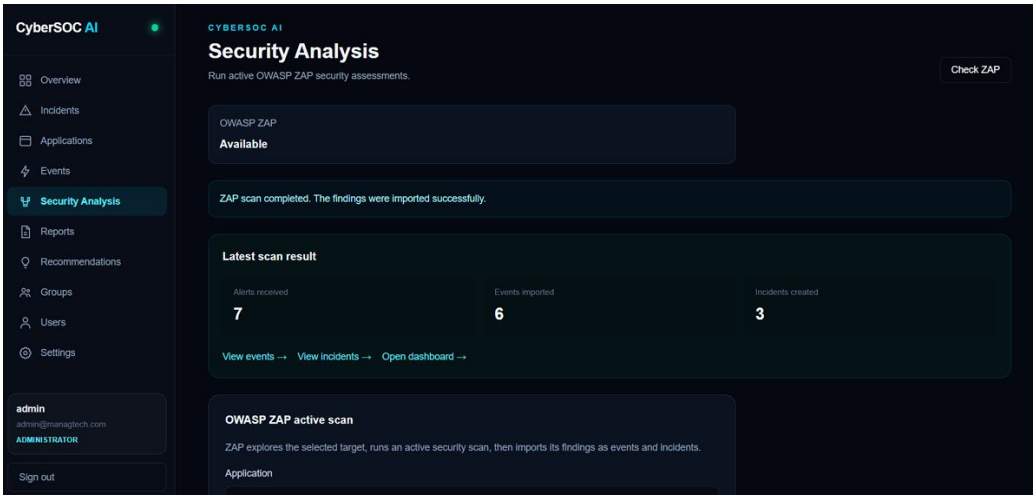


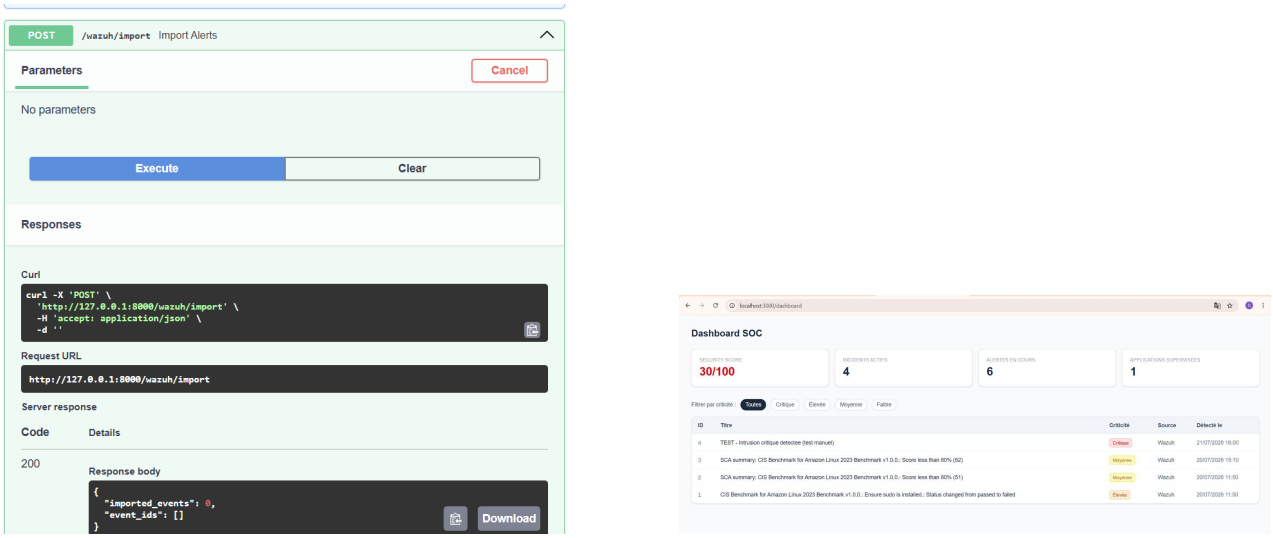
Figure 5.2: OWASP ZAP findings imported during Phase 3 testing

Phase 4 tests covered normal and anomalous SSH behavior, machine-learning incident handling, and the backend metrics endpoint. Gemini recommendations and scheduled reports were not included in the validated test results.

3 Integration Test of the Full Pipeline

To check that the main components worked together, we tested the flow from a Wazuh agent event to its display in CyberSOC. The event was received by Wazuh, imported and normalized by the platform, then used to create an incident and update the Security Score. We checked the resulting data in the dashboard and generated a PDF report from the saved information. This test was carried out in the project environment and was not a long-term production test.

Figure 5.3 shows the Wazuh import and the resulting incidents in the dashboard.



(a) Imported Wazuh events

(b) Incidents displayed in the dashboard

Figure 5.3: Evidence from the end-to-end Wazuh test

4 Detection Tests with Replayed Logs

This section presents two repeatable log-replay checks. The anonymized Fail2ban and authentication records were written to files monitored by Wazuh, then followed through event import and incident display.

4.1 Fail2ban Logs

The script `fixtures/logs/replay_fail2ban.py` writes anonymized Fail2ban lines into the file monitored by Wazuh. It can be run with `python replay_fail2ban.py -input <source> -output <watched-file> -delay 0.1 -limit 100`. The delay makes the input arrive gradually instead of being copied all at once. The script reports each written line and the final number processed. The replayed records were then checked in the incident list and dashboard. This made it possible to follow the input through Wazuh normalization and incident creation, including the address and jail associated with a finding.

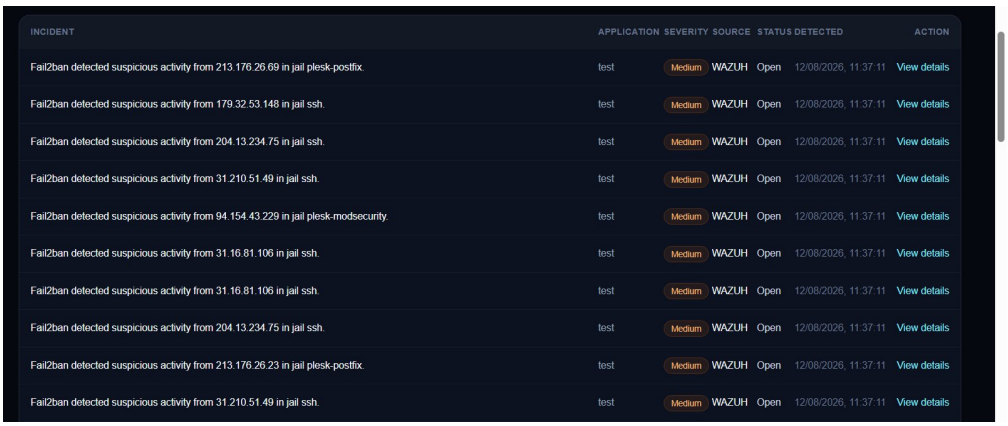


Figure 5.4: Incidents displayed after Fail2ban log replay

4.2 Authentication Logs

The script `fixtures/logs/replay_logs.py` replays anonymized authentication records into the file monitored by Wazuh. It accepts input and output paths, a delay, and an optional line limit. The resulting Wazuh alerts are retrieved by the backend, converted into events, and grouped into incidents. These include failed SSH authentications, login failures, and attempts to log in with a non-existent user. The incident list in Figure 5.5 shows the results of the replay and confirms that event details were retained.

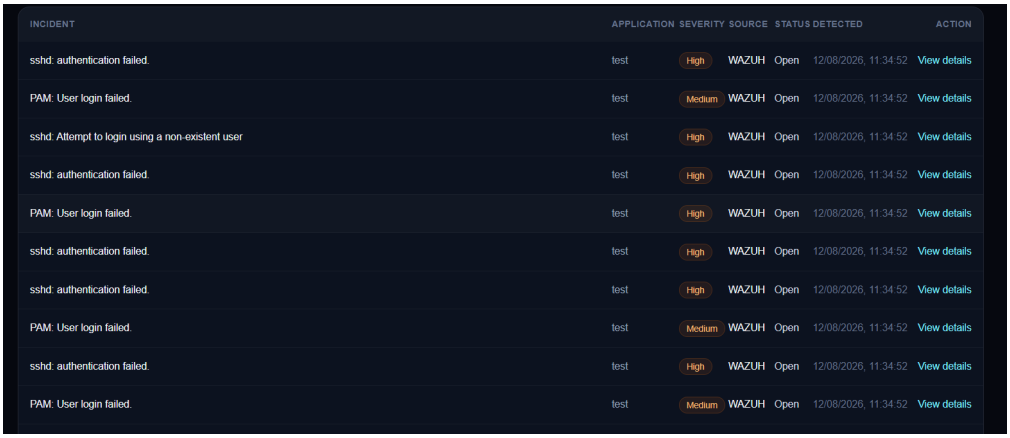


Figure 5.5: Authentication incidents displayed after log replay

5 Tests of the Machine Learning Engine

We tested the machine-learning engine's event collection, anomaly detection, and incident integration. The tests checked that each source collector avoids rereading events, that the model distinguishes normal from unusual behavior, and that detected anomalies are linked to incidents.

Collectors. Wazuh, Suricata, and Docker collectors were each tested with three reads. The first read collected available records, the second found no duplicates, and a later read detected newly added data. Table 5.1 summarizes the results.

Table 5.1: Incremental behavior of the source collectors

Source	First read	Second read	New data
Wazuh	431 events	0	1 event detected
Suricata	23 events	0	1 event detected
Docker	10 records	0	Cursor updated

Prediction and incident integration. The model was tested with normal and unusual behavior. Additional checks confirmed that an anomaly can create or enrich an incident without creating duplicates, and that the incident can be associated with an application. The unit tests were run with `python -m pytest ml-engine/tests`.

End-to-end test. In the complete test, the engine processed 66 SSH events and formed 9 behavioral windows. It identified one window as anomalous while classifying the others as normal. The backend received the anomaly and created a Medium-severity incident with a risk score of 6 for the mapped application.

6 Security and Reliability Tests

This section summarizes checks of role permissions and duplicate handling. Together, these checks confirm that protected actions depend on the user role and that repeated inputs do not create duplicate records.

6.1 Access Control

The access control was verified at two levels. At the API level, a request without a token is rejected with a 401 error, and a request made with an insufficient role is rejected with a 403 error. At the interface level, each role sees only the actions it is allowed to perform: the reader has read-only pages, the analyst does not access the administration options, and only the administrator manages the users, the groups and the deletion of applications. These behaviors are shown in the captures of the platform presented in the previous chapter.

6.2 Duplicate Handling

Two mechanisms prevent the duplication of data. For the alerts, the unique constraint on the raw identifier makes the import idempotent, as shown by the second import that returns zero event. For the machine learning anomalies, the backend answers that the incident already exists when the same window and the same address are sent again, and it enriches an existing incident instead of creating a second one.

7 Results and Discussion

Table 5.2 summarizes the validation of the main groups of requirements.

Table 5.2: Validation of the requirements

Requirements	Validated by	Status
Applications management (RF01 to RF05)	CRUD tests, applications and groups pages	Applications and groups were checked; APIs, Docker environments, and history were not separately validated in the recorded tests
Collection (RF06, RF07)	Wazuh import, ZAP scan, collectors of the ML engine	Validated
Detection (RF08 to RF13)	Wazuh-agent events, replayed Fail2ban and authentication logs, ML engine scenarios	Validated for the documented Wazuh, replay, and SSH anomaly cases
Classification and score (RF14 to RF16)	Detection, score and SQL tests	Validated
Recommendations (RF17, RF18)	Template routes and incident detail views	Template-based recommendations displayed for the documented incident types; Gemini remains optional
Dashboard (RF19 to RF24)	Administrator, analyst, and reader screens; incident and event views	Validated for the supplied screenshots and available workflows
Reports (RF25 to RF29)	Manual PDF generation and read-only consultation	Manual report generation validated; scheduled reports are outside the verified test evidence
Authentication and roles (RF30 to RF32)	401 and 403 tests, role-based interface	Validated

The tests were conducted in a controlled environment using Kali Linux, an Ubuntu target monitored by a Wazuh agent, and the OWASP Juice Shop container. They confirmed that security activity could be reviewed in CyberSOC and that ZAP findings could be imported. In the recorded ZAP test, 7 alerts were received, 6 events imported, and 3 incidents created.

These results are limited to the tested environment. URL-based application mapping and load performance were not tested. Detection accuracy was only partially evaluated, and the machine-learning model was validated for SSH behavior; Suricata and Docker models remain to be trained.

Conclusion

This chapter presented the platform tests, from event collection to incident display, and checked access control and duplicate handling. Tests with anonymized logs and the machine-learning engine confirmed the main workflows. Validation was limited to the project environment and did not include load testing. The general conclusion summarizes the results and future work.

General Conclusion

Before this project, MARTECH Conseils did not have the CyberSOC platform described in this report. The company needed a way to bring security information from its applications and technical environments into one place, so that events and incidents could be reviewed more easily.

We addressed this need by designing and developing a defensive monitoring platform for multiple applications. We first studied the company context and requirements, then designed the architecture and data model. We implemented the platform through successive development cycles using a FastAPI backend, PostgreSQL, a Next.js dashboard, and security tools including Wazuh, Suricata, Docker log collection, and OWASP ZAP.

The platform brings several security tasks together. It collects and normalizes events, detects and classifies incidents, calculates an application Security Score, provides recommendations, and supports incident review and PDF reporting. It also includes authentication and role-based access for Administrators, SOC Analysts, and Readers. For SSH anomaly detection, the machine-learning service loads a model trained offline and analyzes incoming event windows; it does not retrain for each event.

We tested the main workflows through API checks, dashboard review, Wazuh agent events, anonymized log replay, ZAP scans, and machine-learning scenarios. In the recorded ZAP test, the platform received seven alerts, imported six events, and created three incidents. Kali Linux and an Ubuntu target were used in a controlled test environment, and OWASP Juice Shop provided a deliberately vulnerable web application for security exercises. These tests confirmed the main processing and review workflows in the project environment.

The project provides MARTECH Conseils with a working foundation for centralized security monitoring and demonstrates how several security sources can be connected to a shared platform. It also gave us practical experience in event processing, security-tool integration, access control, machine-learning integration, and testing a multi-service application.

Some work remains before the platform can be considered ready for long-term operational use. End-to-end testing of URL-based application monitoring should be completed. The Security Score and anomaly detector need evaluation with broader data, including measured false-positive and false-negative rates. Performance, reliability, and security-hardening tests should also be carried out. Future work could further calibrate the score, reduce collection delays, improve the context of recommendations, and assess additional roles and security standards. These steps would build on the current platform and help prepare it for wider use.

Bibliography

[1] MARTECH Conseils, "Company information and services." [Online]. Available: https://martech.ma/. Accessed: 24 October 2026.

[2] MARTECH Conseils, "ManagTech: Gestion des marchés." [Online]. Available: https://martech.ma/Gestion-des-march%C3%A9s. Accessed: 24 October 2026.

[3] Wazuh, "How log data collection works." [Online]. Available: https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/how-it-works.html. Accessed: 24 October 2026.

[4] Open Information Security Foundation, "EVE JSON output." [Online]. Available: https://docs.suricata.io/en/latest/output/eve/eve-json-output.html. Accessed: 24 October 2026.

[5] OWASP, "ZAP API." [Online]. Available: https://www.zaproxy.org/docs/api/. Accessed: 24 October 2026.

[6] Docker, "Docker Compose documentation." [Online]. Available: https://docs.docker.com/compose/. Accessed: 24 October 2026.

[7] Scikit-learn developers, "IsolationForest." [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html. Accessed: 24 October 2026.

Image Credits

The following sources identify the tool logos used in Chapter 4.

- Python logo: https://simpleicons.org/?q=python
- FastAPI logo: https://simpleicons.org/?q=fastapi
- Next.js logo: https://simpleicons.org/?q=next.js
- Tailwind CSS logo: https://simpleicons.org/?q=tailwindcss
- PostgreSQL logo: https://simpleicons.org/?q=postgresql
- Docker logo: https://simpleicons.org/?q=docker
- Wazuh logo: https://wazuh.com/brand-assets/
- Suricata logo: https://suricata.io/branding-images/
- OWASP ZAP logo: https://commons.wikimedia.org/wiki/File:OWASP_ZAP_logo.svg
- Scikit-learn logo: https://simpleicons.org/?q=scikitlearn
- Prometheus logo: https://simpleicons.org/?q=prometheus
- Grafana logo: https://simpleicons.org/?q=grafana

Appendices: Project Files

The project documents, diagrams, and test images are organized in the following Google Drive folders.

- `00_Project_Organization`: methodology, internship schedule, and team task distribution.
- `01_Phase_0_Study_and_Requirements`: project study, requirements, and use-case diagrams.
- `02_Phase_1_Architecture_and_Setup`: architecture, diagrams, data design, and setup documents.
- `03_Phase_2_MVP_Core`: MVP documents and related test evidence.
- `04_Phase_3_Security_Integrations`: security integrations, platform screenshots, and test evidence.
- `05_Phase_4_AI_and_Monitoring`: machine-learning diagrams and test results.
- `06_Additional_Images_From_Reference_Report`: image files from the supplied reference report.